

Analysis of Large Graphs: Community Detection

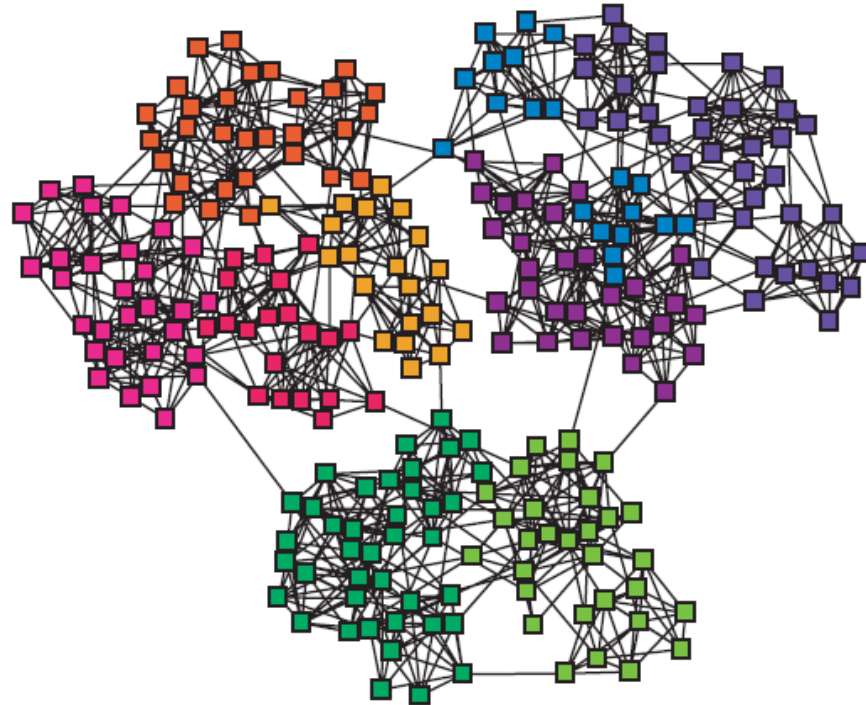
Adapted from chapter 10 in
Mining of Massive Datasets

Jure Leskovec, Anand Rajaraman, Jeff Ullman Stanford University

<http://www.mmds.org>

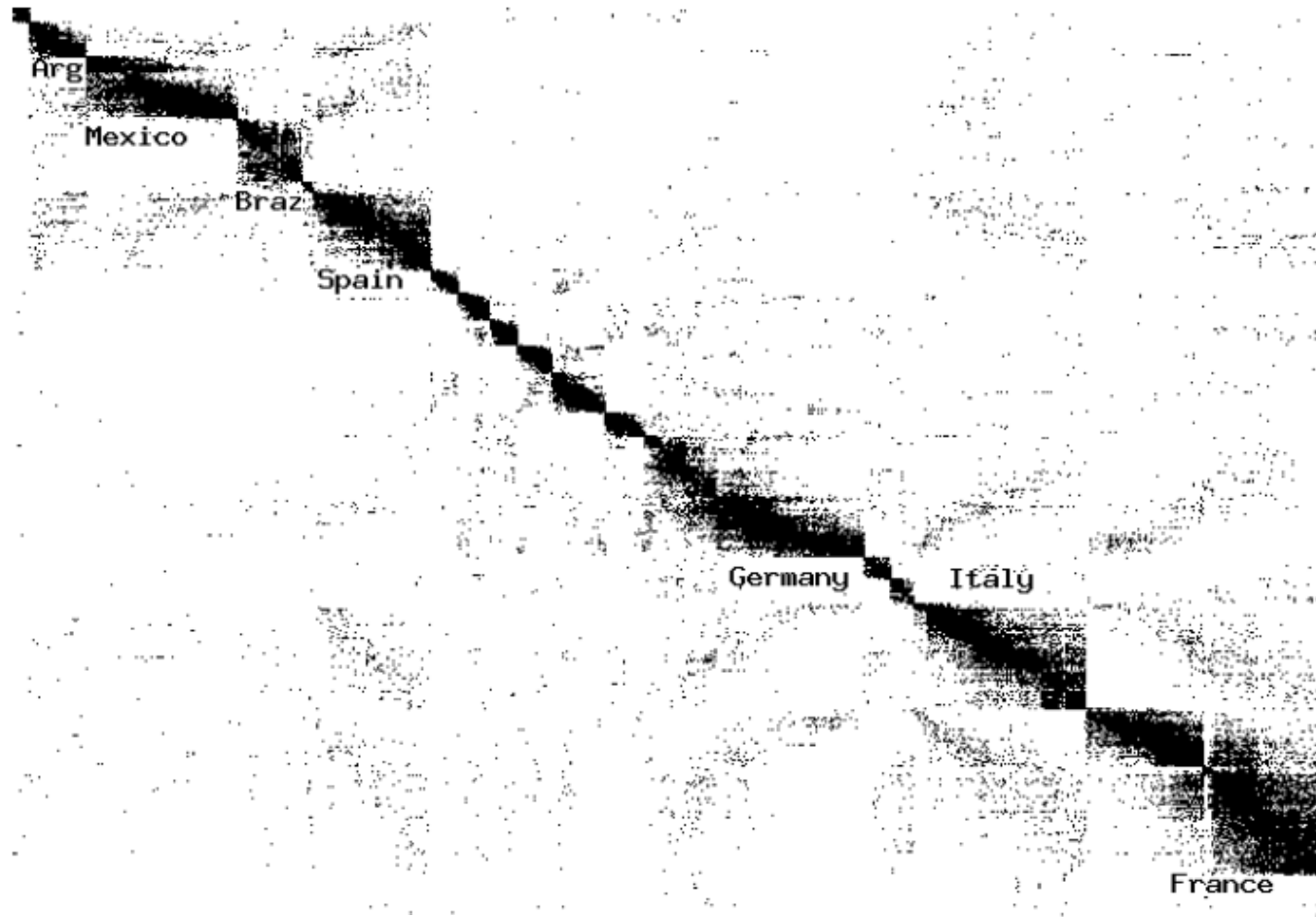
Networks & Communities

- We often think of networks being organized into **modules, cluster, communities**:
- Goal: Find Densely Linked Clusters



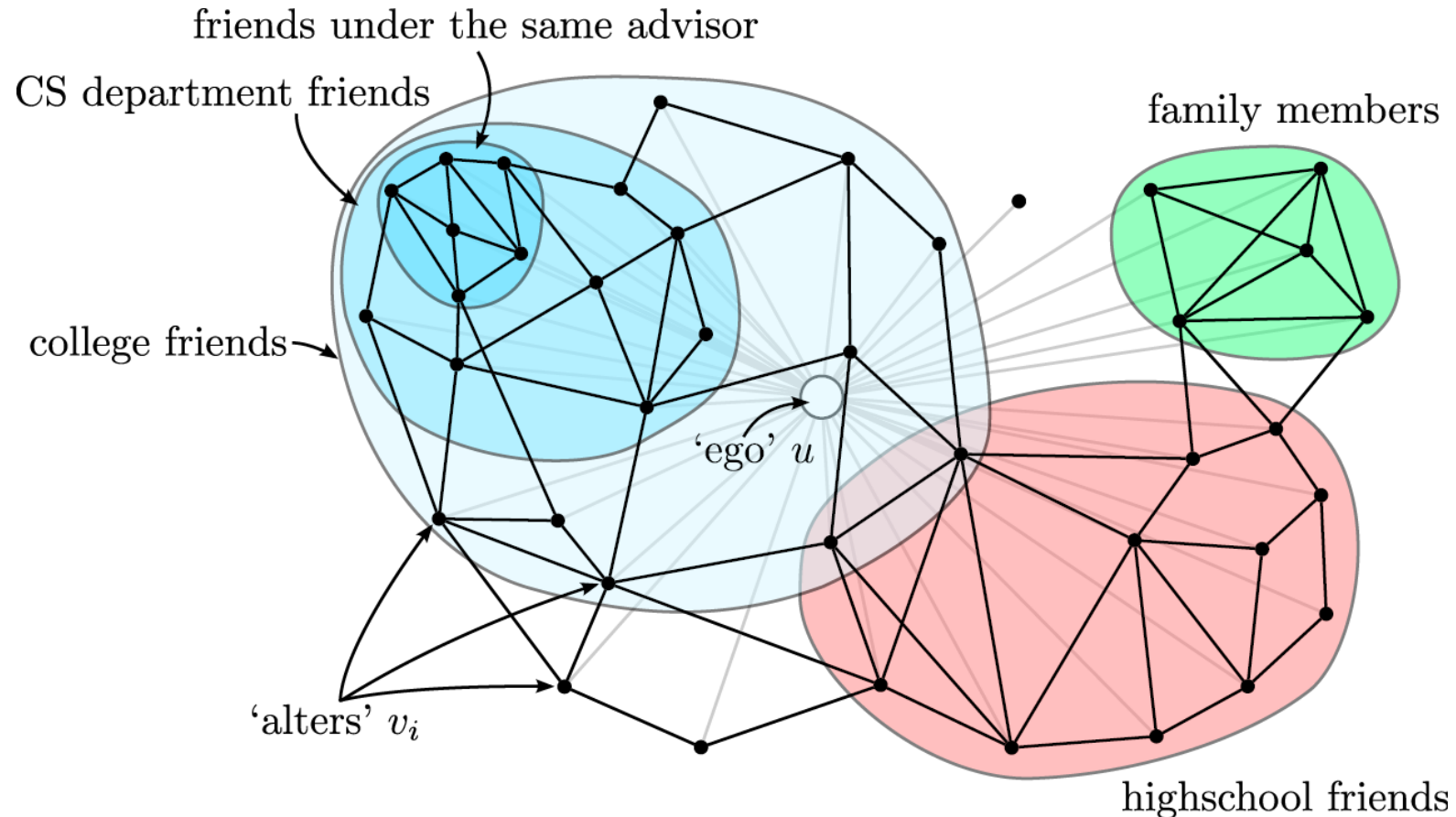
Movies and Actors

- Clusters in Movies-to-Actress graph:



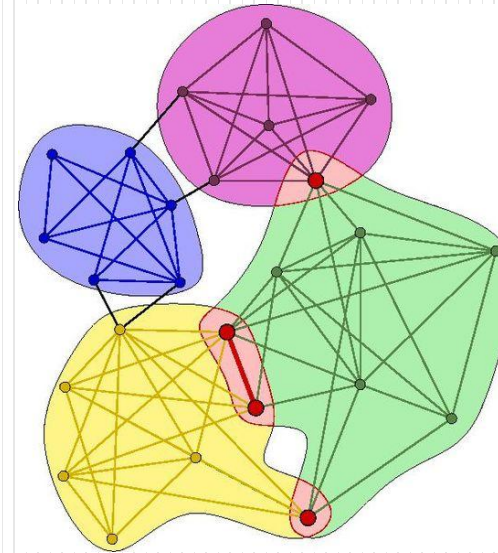
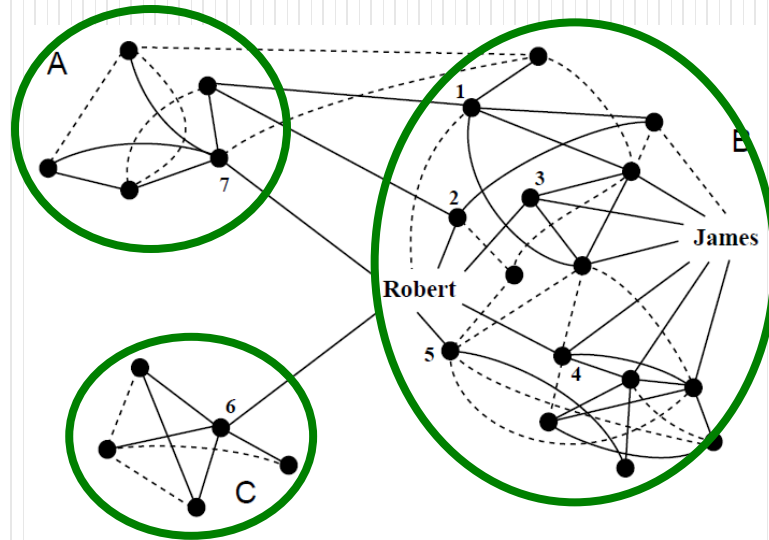
Twitter & Facebook

- **Discovering social circles, circles of trust:**



Community Detection

How to find communities?

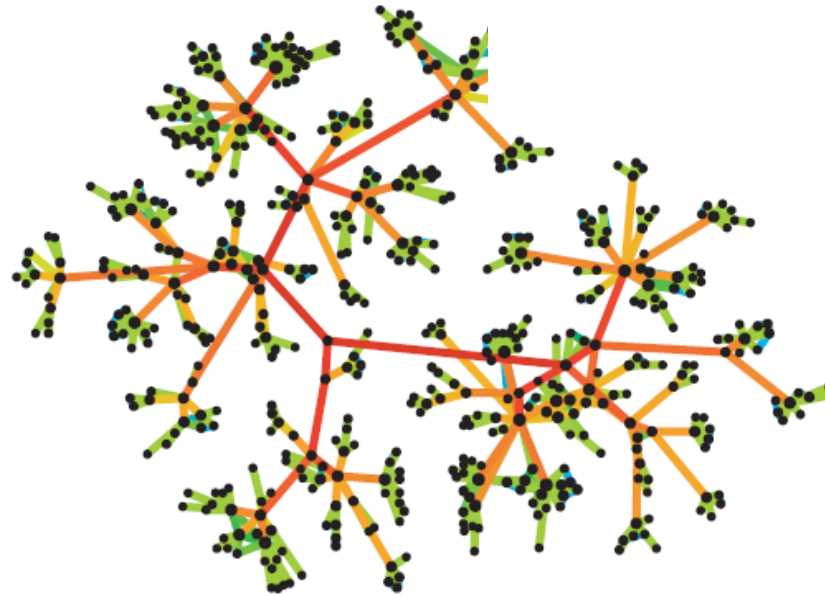
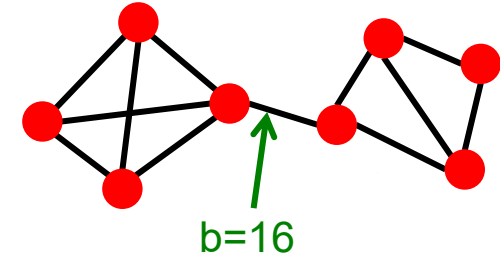


Crisp partitioning

Hierachical division

Method 1: Strength of Weak Ties

- **Edge betweenness:** Number of shortest paths passing over the edge
- **Intuition:**

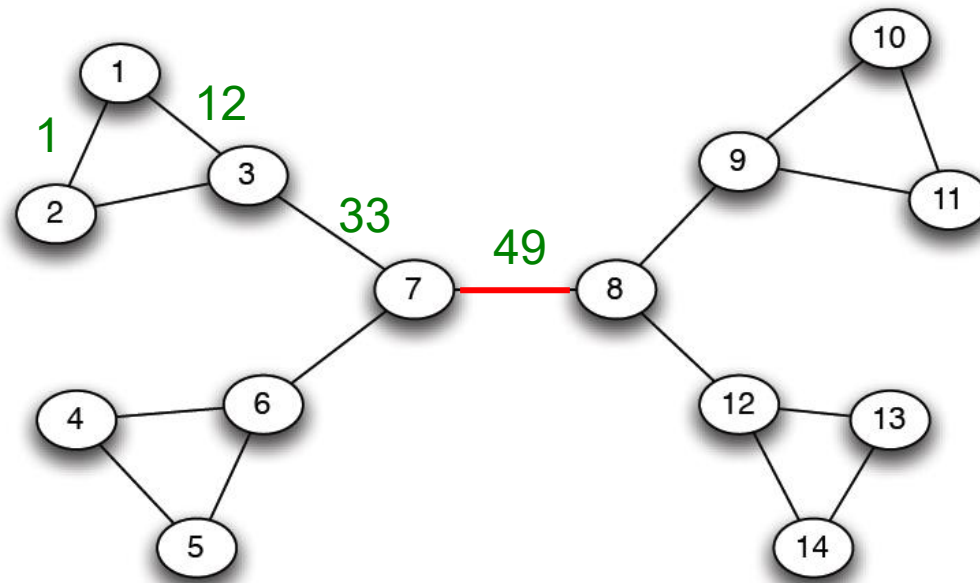


Edge betweenness
in a real network

Method: Girvan-Newman

- For undirected unweighted graphs
- IDEA: Divisive hierarchical clustering based on the notion of edge **betweenness**:
 - **Compute betweenness with Girvan-Newman Algorithm**
- **Algorithm:**
 - **Repeat until no edges are left:**
 - Calculate betweenness of edges
 - Remove edges with highest betweenness
 - Connected components are communities
 - Gives a hierarchical decomposition of the network

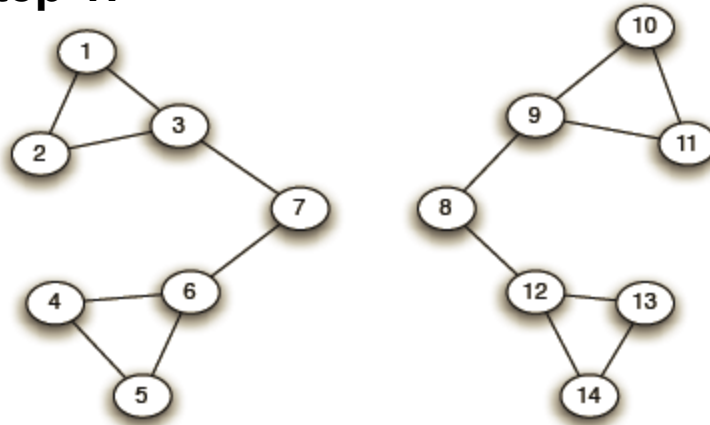
Girvan-Newman: Example



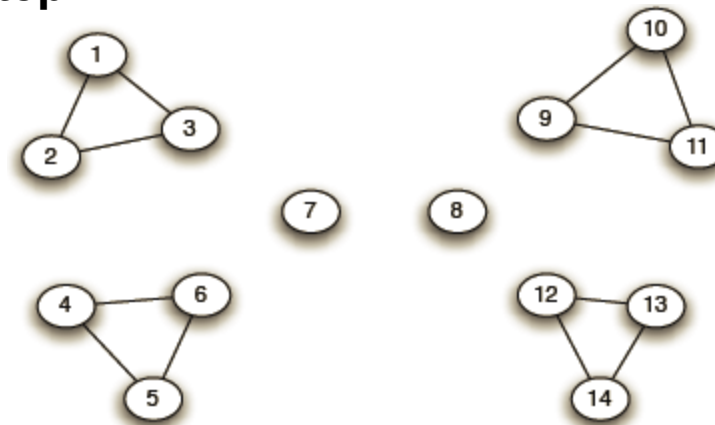
Need to re-compute
betweenness at
every step

Girvan-Newman: Example

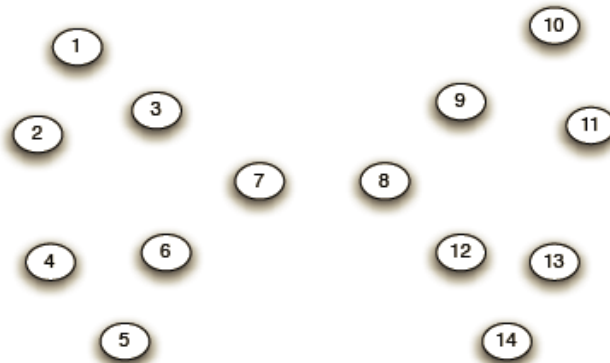
Step 1:



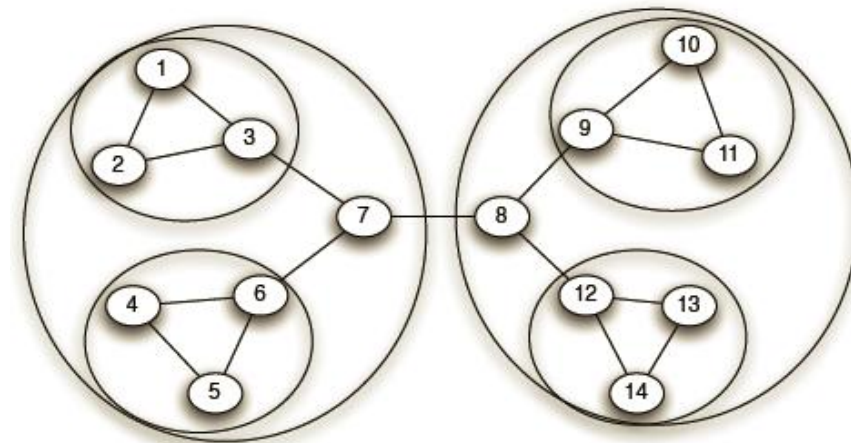
Step 2:



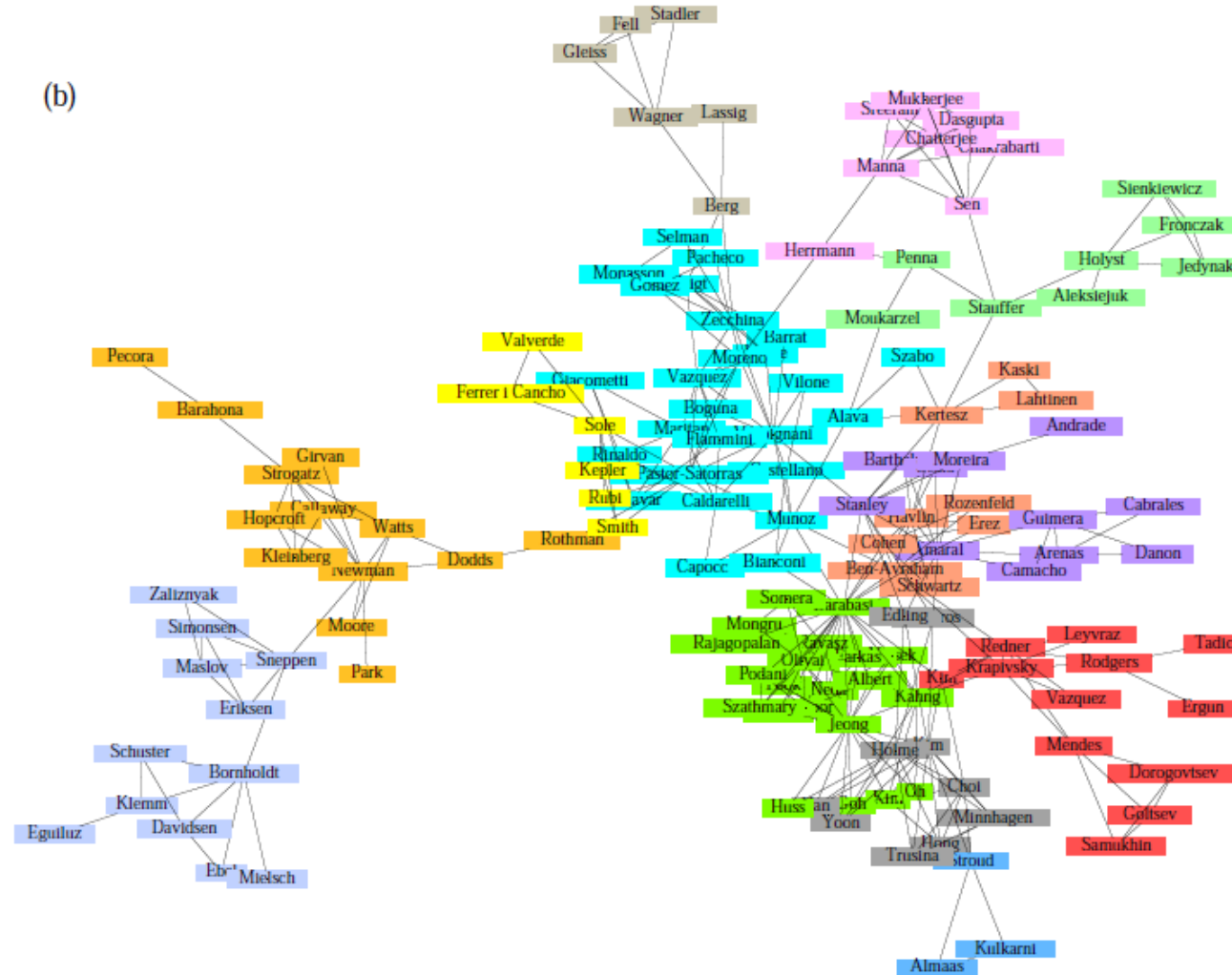
Step 3:



Hierarchical network decomposition:

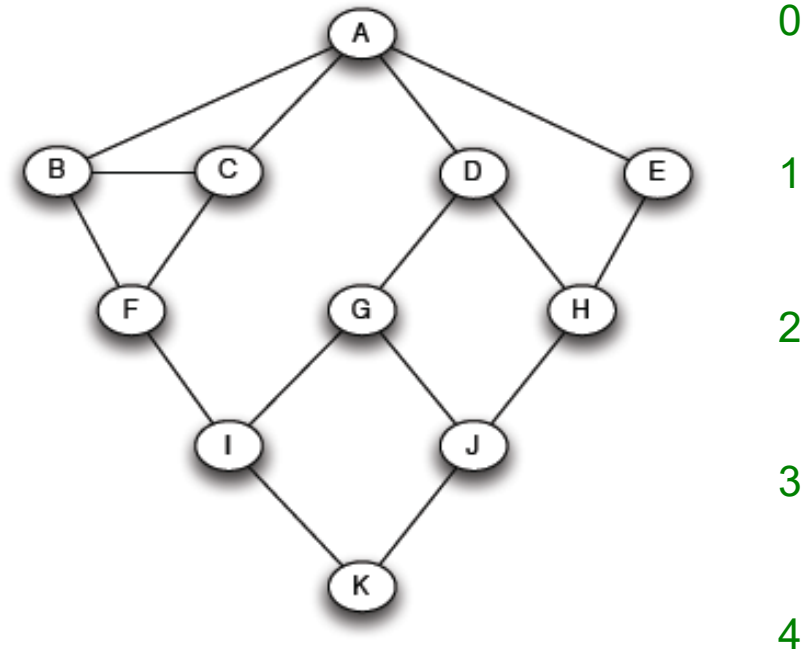
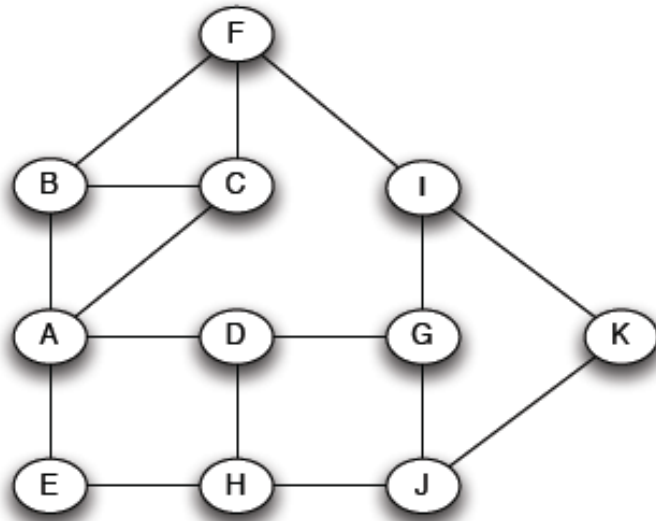


Girvan-Newman: Results



How to Compute Betweenness?

- Compute shortest paths starting from each node in turn
- Start with A
- Breadth first search



0

1

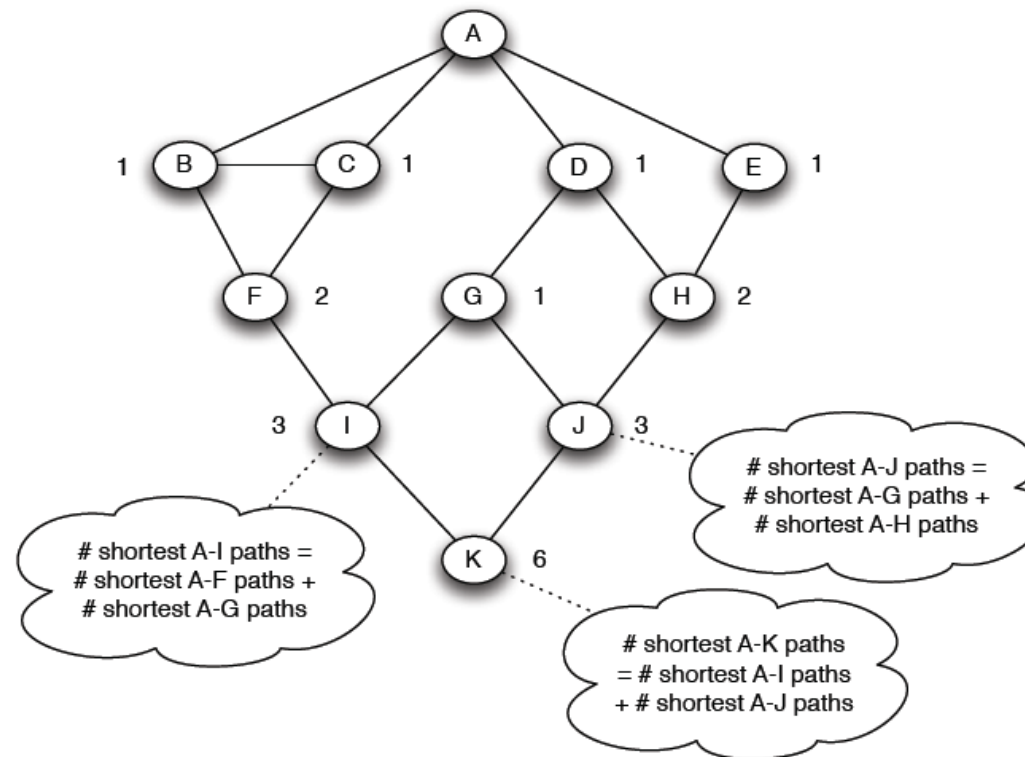
2

3

4

How to Compute Betweenness?

- Count the number of shortest paths from *A* to each of the other nodes of the network:



How to Compute Betweenness?

- **Compute betweenness by working up the tree:** If there are multiple paths count them fractionally

The algorithm:

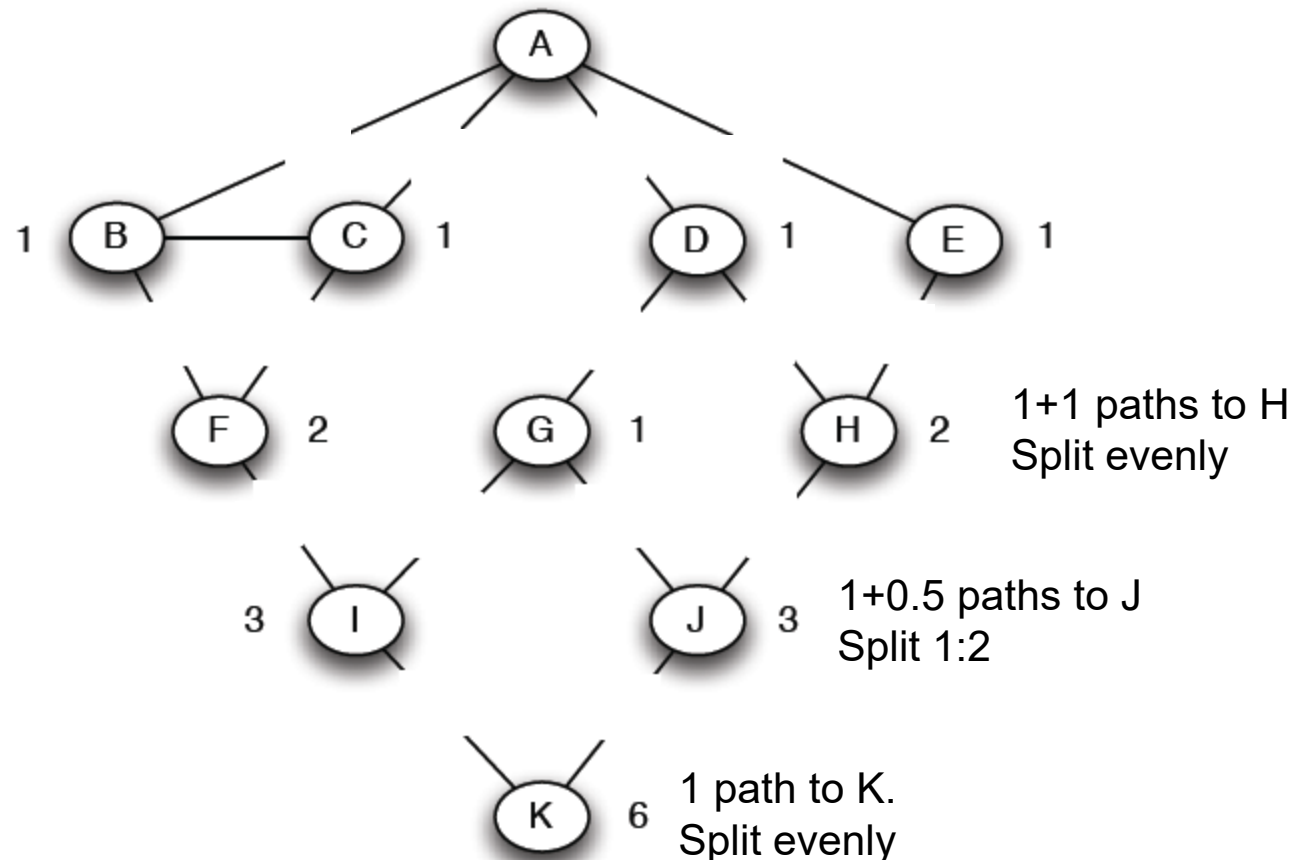
• Add edge flows:

-- node flow =

$1 + \sum \text{child edges}$

-- split the flow up based on the parent value

• Repeat the BFS procedure for each starting node U



How to Compute Betweenness?

- **Compute betweenness by working up the tree:** If there are multiple paths count them fractionally

Time complexity of G-N:
O(ne)

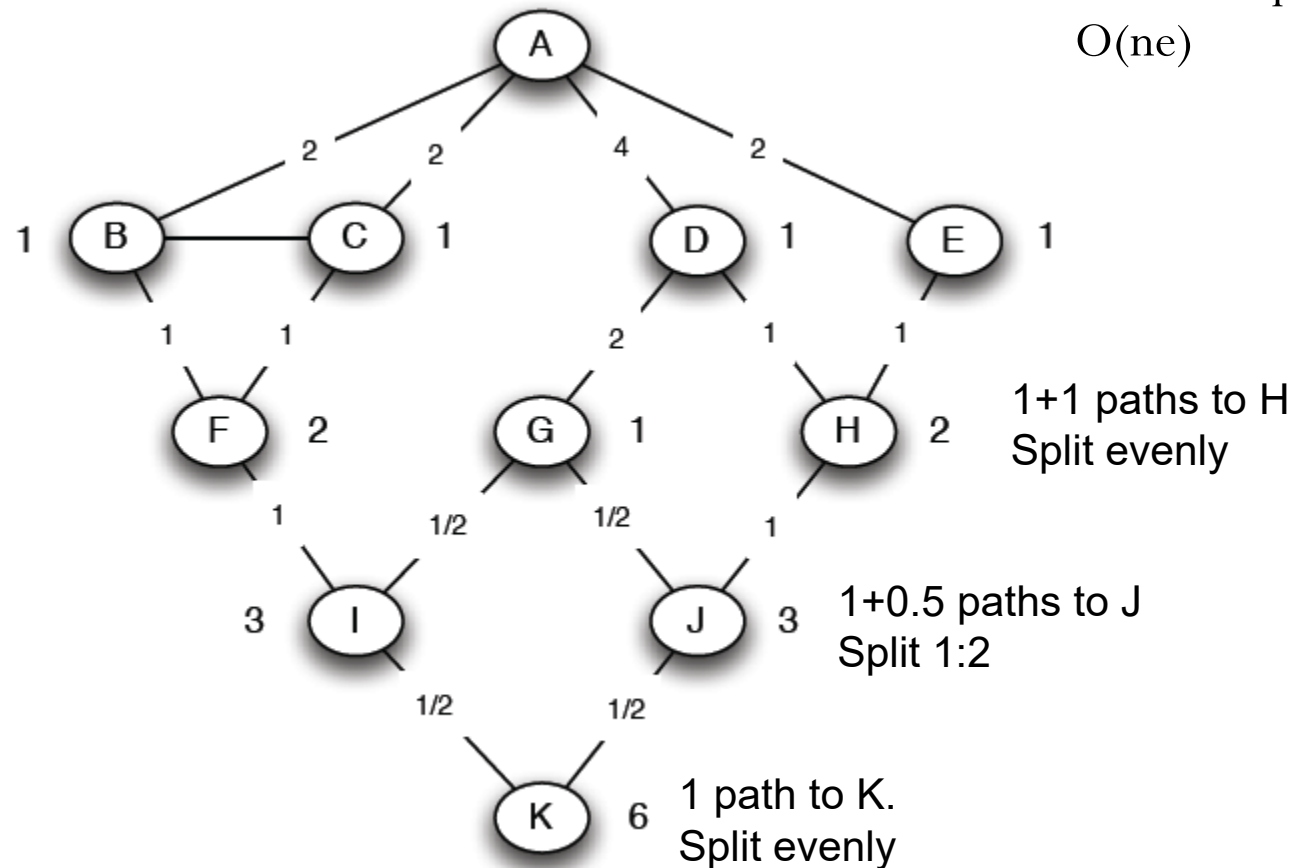
The algorithm:

- Add edge **flows**:

```
-- node flow = 1 +  $\sum$  child edges
```

- split the flow up based on the parent value

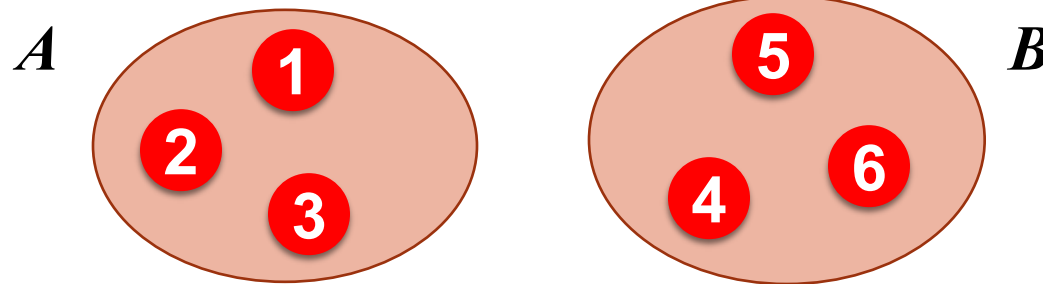
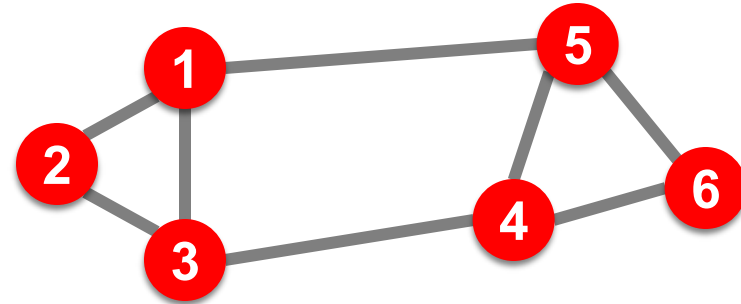
- Repeat the BFS procedure for each starting node U



Spectral Clustering

Graph Partitioning – as an optimization problem

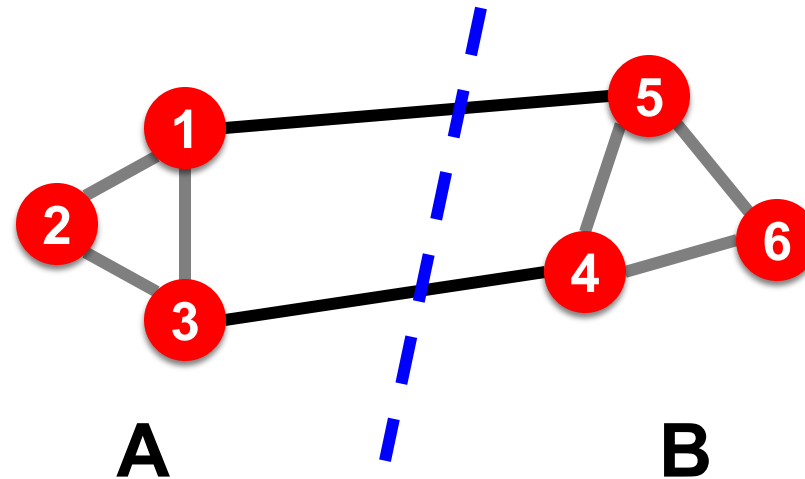
- Undirected graph $G(V, E)$:
- **Bi-partitioning task:**
 - Divide vertices into two disjoint groups A, B



- **Questions:**
 - How can we define a “good” partition of G ?
 - How can we efficiently identify such a partition?

Graph Partitioning

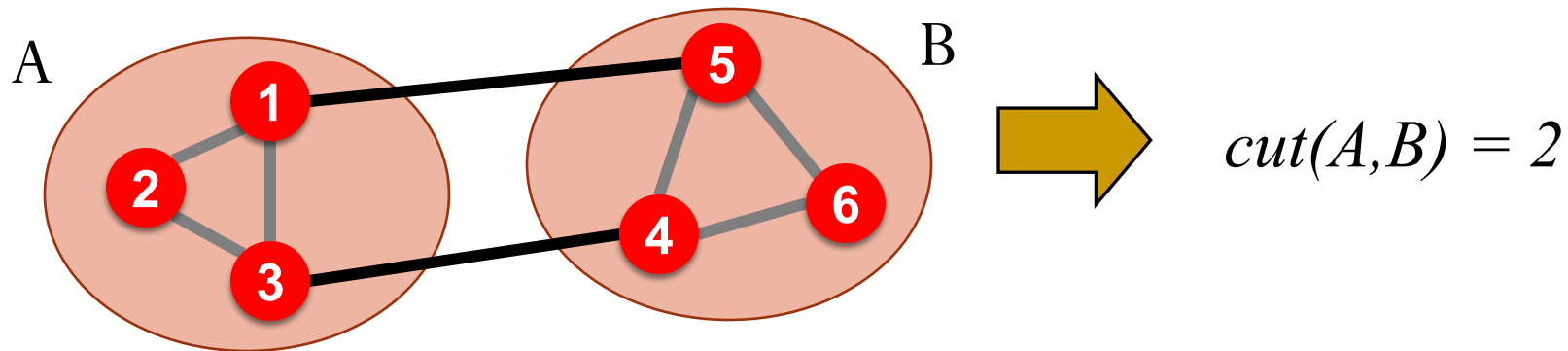
- **What makes a good partition?**
 - Maximize the number of within-group connections
 - Minimize the number of between-group connections



Graph Cuts

- Express partitioning objectives as a function of the “edge cut” of the partition
- **Cut:** Set of edges with only one vertex in a group:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}$$

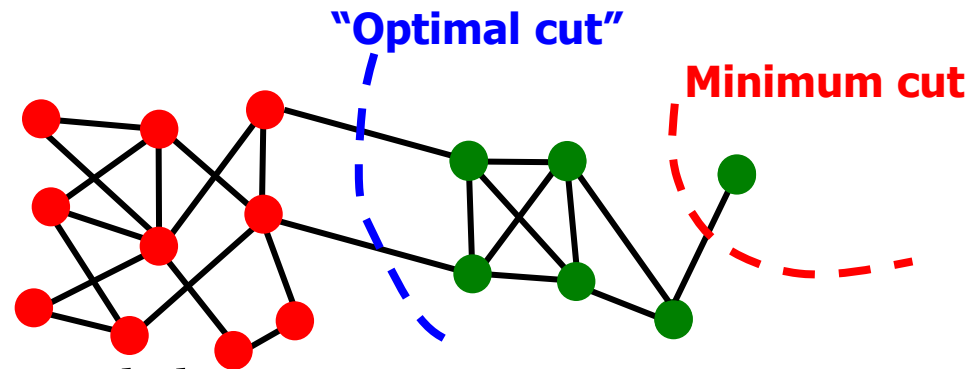


Graph Cut Criterion

- **Criterion: Minimum-cut**
 - Minimize weight of connections between groups

$$\arg \min_{A,B} \text{cut}(A,B)$$

- **Degenerate case:**



- **Problem:**
 - Only considers external cluster connections
 - Does not consider internal cluster connectivity

Graph Cut Criteria

- **Criterion: Normalized-cut** [Shi-Malik, '97]
 - Connectivity between groups relative to the density of each group

$$ncut(A, B) = \frac{cut(A, B)}{vol(A)} + \frac{cut(A, B)}{vol(B)}$$

vol(A): total weight of the edges with at least one endpoint in ***A***

- **Why use this criterion?**
 - Produces more balanced partitions
- **How do we efficiently find a good partition?**
 - **Problem:** Computing optimal cut is NP-hard

Spectral Graph Partitioning

- A : adjacency matrix of undirected G
 - $A_{ij} = 1$ if (i, j) is an edge, else 0
- x is a vector in $\mathbb{R}^n$ with components $(x_1, \dots, x_n)$
 - Think of it as a label/value of each node of G
- **What is the meaning of $A \cdot x$?**

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$y_i = \sum_{j=1}^n A_{ij} x_j = \sum_{(i,j) \in E} x_j$$

- **Entry y_i is a sum of labels x_j of neighbors of i**

What is the meaning of Ax ?

- j^{th} coordinate of $A \cdot x$:

- Sum of the x -values of neighbors of j
- Make this a new value at node j

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

$$A \cdot x = \lambda \cdot x$$

- **Spectral Graph Theory:**

- Analyze the “spectrum” of matrix representing G
- **Spectrum:** Eigenvectors x_i of a graph, ordered by the magnitude (strength) of their corresponding eigenvalues λ_i :

$$\Lambda = \{\lambda_1, \lambda_2, \dots, \lambda_n\}$$

$$\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$$

Example: d-regular graph

- Suppose all nodes in G have degree d and G is connected
- What are some eigenvalues/vectors of G ? $A \cdot x = \lambda \cdot x$

What is λ ? What is x ?

- Let's try: $x = (1, 1, \dots, 1)$
- Then: $A \cdot x = (d, d, \dots, d) = \lambda \cdot x$. So: $\lambda = d$
- We found eigenpair of G : $x = (1, 1, \dots, 1), \lambda = d$

Example: Graph on 2 components

- What if G is not connected?

- G has 2 components, each d -regular

- What are some eigenvectors?

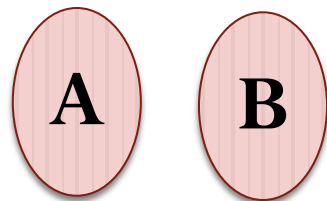
- x = Put all **1**s on A and **0**s on B or vice versa

- $x' = (\underbrace{1, \dots, 1}_{|A|}, \underbrace{0, \dots, 0}_{|B|})$ then $A \cdot x' = (d, \dots, d, 0, \dots, 0)$

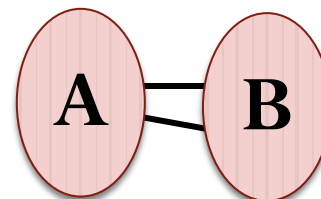
- $x'' = (\underbrace{0, \dots, 0}_{|A|}, \underbrace{1, \dots, 1}_{|B|})$ then $A \cdot x'' = (0, \dots, 0, d, \dots, d)$

- And so in both cases the corresponding $\lambda = d$

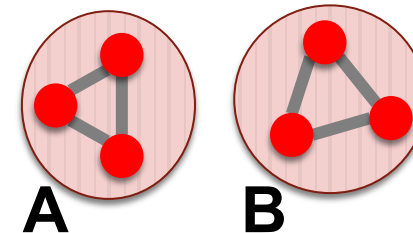
- A bit of intuition:



$$\lambda_n = \lambda_{n-1}$$



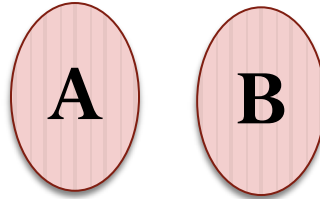
$$\lambda_n - \lambda_{n-1} \approx 0$$



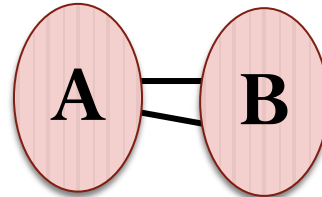
2nd largest eigval.
 λ_{n-1} now has
value very close
to λ_n

More Intuition

More intuition:



$$\lambda_n = \lambda_{n-1}$$



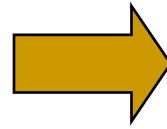
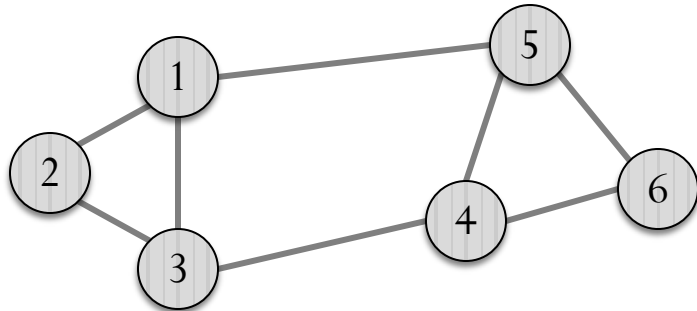
$$\lambda_n - \lambda_{n-1} \approx 0$$

2nd largest eigval.
 λ_{n-1} now has
value very close
to λ_n

- If the graph is connected (right example) then we already know that $\mathbf{x}_n = (\mathbf{1}, \dots \mathbf{1})$ is an eigenvector
- Since eigenvectors are orthogonal then the components of $\mathbf{x}_{n-1}$ sum to $\mathbf{0}$.
 - Why? Because $\mathbf{x}_n \cdot \mathbf{x}_{n-1} = \sum_i \mathbf{x}_n[i] \cdot \mathbf{x}_{n-1}[i]$
- So we can look at the eigenvector of the 2nd largest eigenvalue and declare nodes with positive label in **A** and negative label in **B**.
- **But there is still lots to sort out.**

Matrix Representations

- **Adjacency matrix (A):**
 - $n \times n$ matrix
 - $A=[a_{ij}]$, $a_{ij}=1$ if edge between node i and j

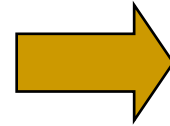
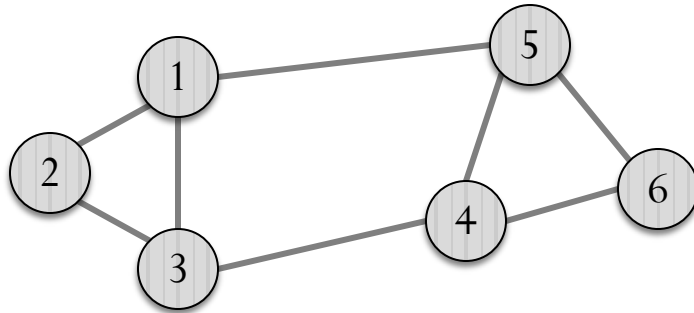


	1	2	3	4	5	6
1	0	1	1	0	1	0
2	1	0	1	0	0	0
3	1	1	0	1	0	0
4	0	0	1	0	1	1
5	1	0	0	1	0	1
6	0	0	0	1	1	0

- **Important properties:**
 - Symmetric matrix
 - Eigenvectors are real and orthogonal

Matrix Representations

- **Degree matrix (D):**
 - $n \times n$ diagonal matrix
 - $D=[d_{ii}]$, d_{ii} = degree of node i

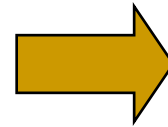
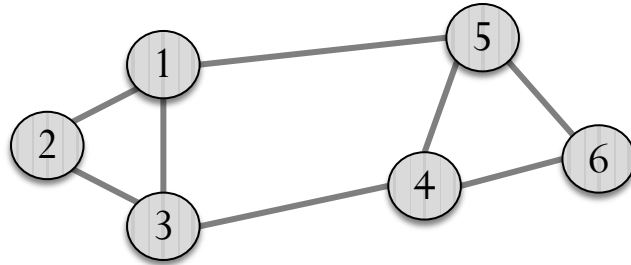


	1	2	3	4	5	6
1	3	0	0	0	0	0
2	0	2	0	0	0	0
3	0	0	3	0	0	0
4	0	0	0	3	0	0
5	0	0	0	0	3	0
6	0	0	0	0	0	2

Matrix Representations

- **Laplacian matrix (L):**

- $n \times n$ symmetric matrix



	1	2	3	4	5	6
1	3	-1	-1	0	-1	0
2	-1	2	-1	0	0	0
3	-1	-1	3	-1	0	0
4	0	0	-1	3	-1	-1
5	-1	0	0	-1	3	-1
6	0	0	0	-1	-1	2

$$L = D - A$$

- **What is trivial eigenpair?**

- $\mathbf{x} = (1, \dots, 1)$ then $L \cdot \mathbf{x} = \mathbf{0}$ and so $\lambda = \lambda_1 = 0$

- **Important properties:**

- **Eigenvalues** are non-negative real numbers
- **Eigenvectors** are real and orthogonal

λ_2 as optimization problem

- **Fact: For symmetric matrix M :**

$$\lambda_2 = \min_x \frac{x^T M x}{x^T x}$$

Rayleigh quotient

- **What is the meaning of $\min x^T L x$ on G ?**

- $x^T L x = \sum_{i,j=1}^n L_{ij} x_i x_j = \sum_{i,j=1}^n (D_{ij} - A_{ij}) x_i x_j$
- $= \sum_i D_{ii} x_i^2 - \sum_{(i,j) \in E} 2x_i x_j$
- $= \sum_{(i,j) \in E} (\underbrace{x_i^2 + x_j^2}_{\text{green brace}} - 2x_i x_j) = \sum_{(i,j) \in E} (x_i - x_j)^2$

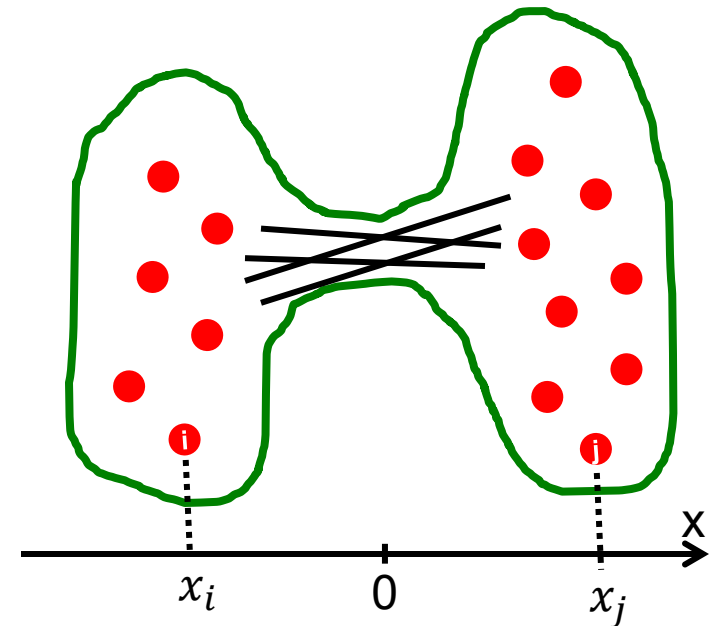
λ_2 as optimization problem

- **What else do we know about $\mathbf{x}$?**

- $\mathbf{x}$ is unit vector: $\sum_i x_i^2 = 1$
- $\mathbf{x}$ is orthogonal to 1st eigenvector $(1, \dots, 1)$ thus: $\sum_i x_i \cdot 1 = \sum_i x_i = 0$

- **Remember:**

$$\lambda_2 = \min_{\substack{\text{All labelings} \\ \text{of nodes } i \text{ so} \\ \text{that } \sum x_i = 0}} \frac{\sum_{(i,j) \in E} (x_i - x_j)^2}{\sum_i x_i^2}$$



Balance to minimize

Find Optimal Cut [Fiedler'73]

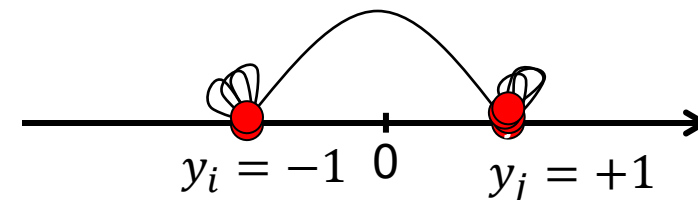
- Back to finding the optimal cut
- Express partition (A,B) as a vector

$$y_i = \begin{cases} +1 & \text{if } i \in A \\ -1 & \text{if } i \in B \end{cases}$$

- We can minimize the cut of the partition by finding a non-trivial vector y that **minimizes**:

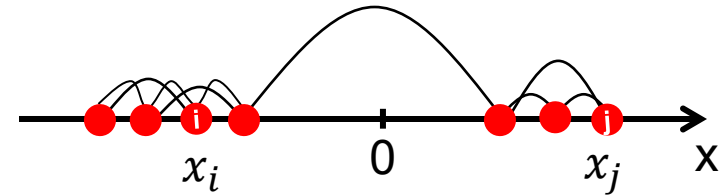
$$\arg \min_{y \in [-1, +1]^n} f(y) = \sum_{(i,j) \in E} (y_i - y_j)^2$$

Can't solve exactly. Let's relax y and allow it to take any real value.



Rayleigh Theorem

$$\min_{y \in \mathbb{R}^n} f(y) = \sum_{(i,j) \in E} (y_i - y_j)^2 = y^T L y$$



- ■ $\lambda_2 = \min_y f(y)$: The minimum value of $f(y)$ is given by the 2nd smallest eigenvalue λ_2 of the Laplacian matrix L
- $x = \arg \min_y f(y)$: The optimal solution for y is given by the corresponding eigenvector x , referred as the **Fiedler vector**

So far...

- **How to define a “good” partition of a graph?**
 - Minimize a given graph cut criterion
- **How to efficiently identify such a partition?**
 - Approximate using information provided by the eigenvalues and eigenvectors of a graph
- **Spectral Clustering**

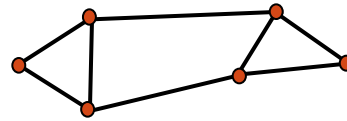
Spectral Clustering Algorithms

- **Three basic stages:**
 - **1) Pre-processing**
 - Construct a matrix representation of the graph
 - **2) Decomposition**
 - Compute eigenvalues and eigenvectors of the matrix
 - Map each point to a lower-dimensional representation based on one or more eigenvectors
 - **3) Grouping**
 - Assign points to two or more clusters, based on the new representation

Spectral Partitioning Algorithm

- 1) **Pre-processing:**

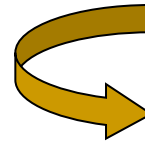
- Build Laplacian matrix $\mathbf{L}$ of the graph



	1	2	3	4	5	6
1	3	-1	-1	0	-1	0
2	-1	2	-1	0	0	0
3	-1	-1	3	-1	0	0
4	0	0	-1	3	-1	-1
5	-1	0	0	-1	3	-1
6	0	0	0	-1	-1	2

- 2) **Decomposition:**

- Find eigenvalues λ and eigenvectors $\mathbf{x}$ of the matrix $\mathbf{L}$
- Map vertices to corresponding components of λ_2



$\lambda =$

0.0
1.0
3.0
3.0
4.0
5.0

$\mathbf{X} =$

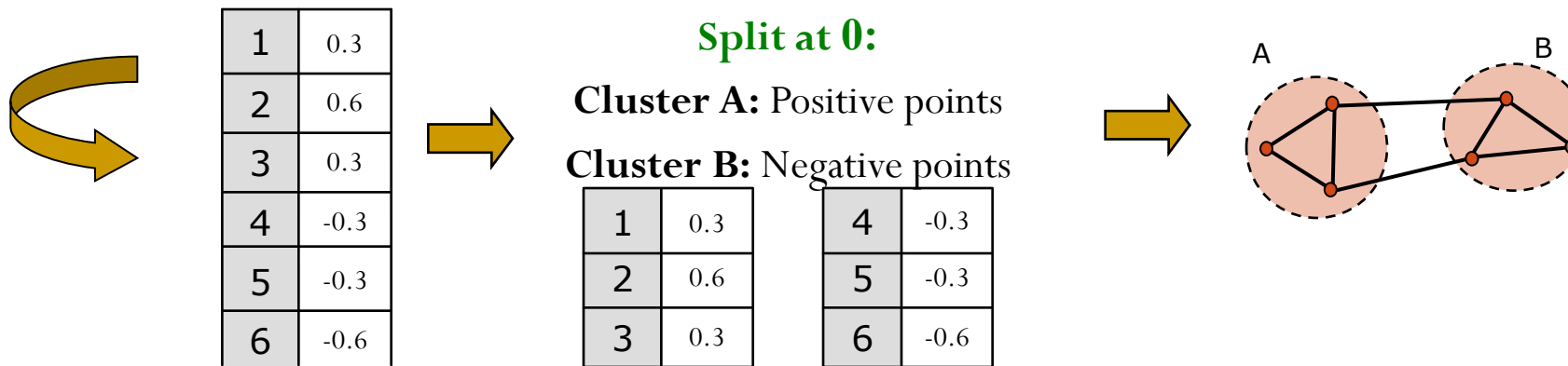
0.4	0.3	-0.5	-0.2	-0.4	-0.5
0.4	0.6	0.4	-0.4	0.4	0.0
0.4	0.3	0.1	0.6	-0.4	0.5
0.4	-0.3	0.1	0.6	0.4	-0.5
0.4	-0.3	-0.5	-0.2	0.4	0.5
0.4	-0.6	0.4	-0.4	-0.4	0.0

1	0.3
2	0.6
3	0.3
4	-0.3
5	-0.3
6	-0.6

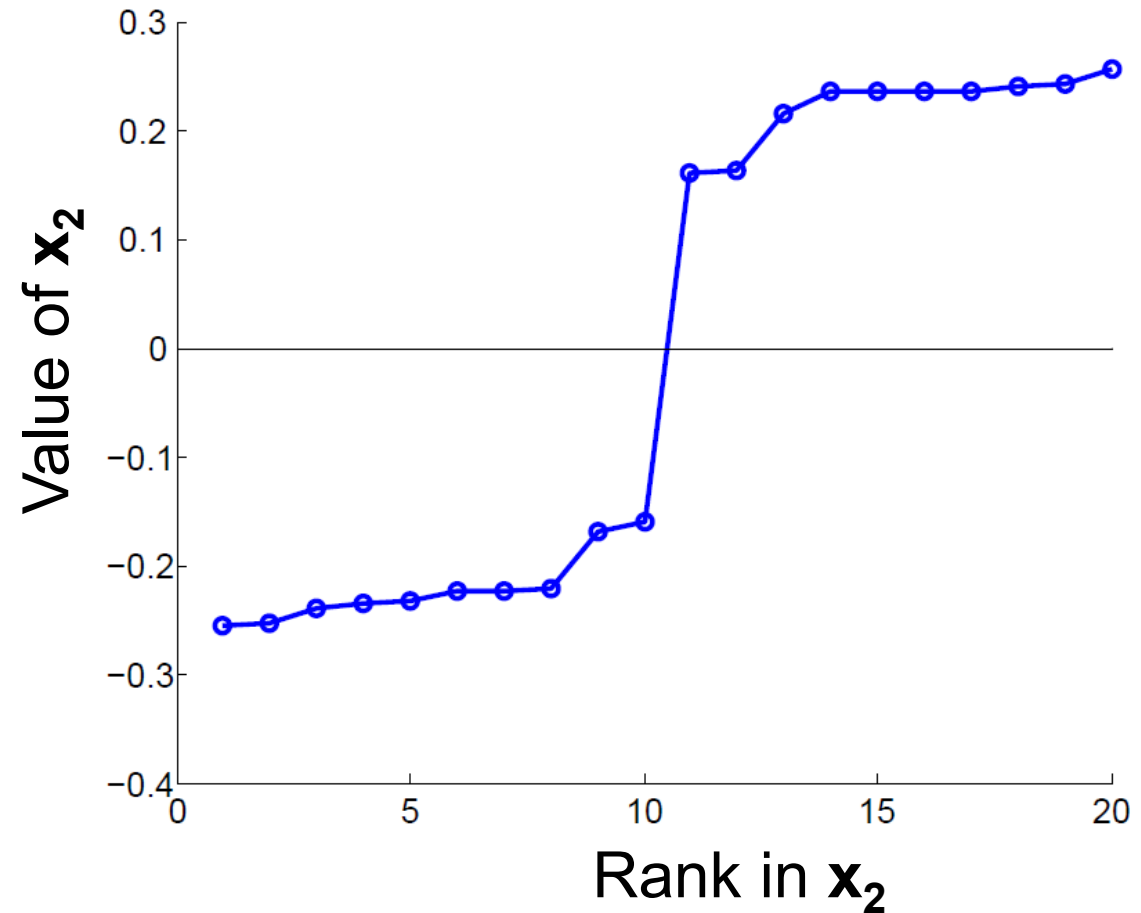
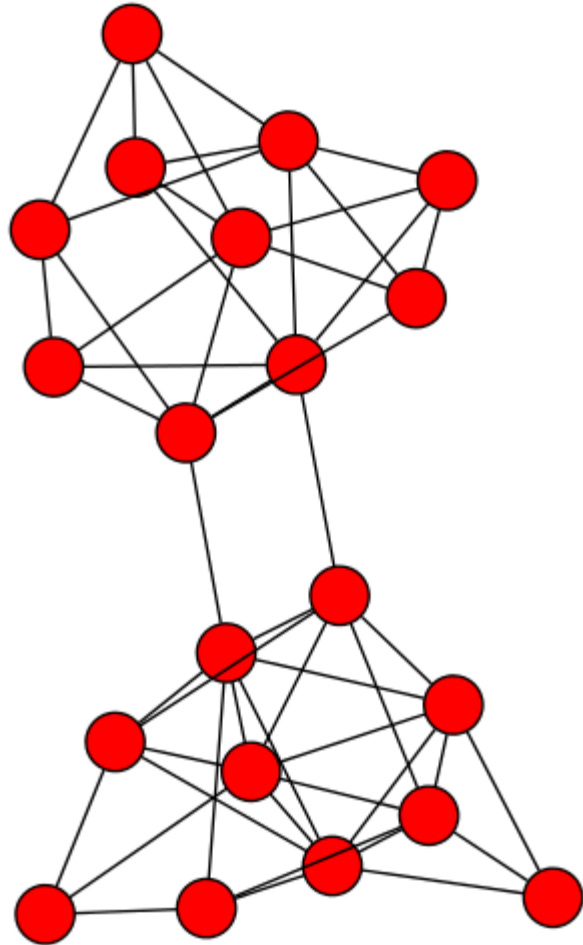
How do we now find the clusters?

Spectral Partitioning

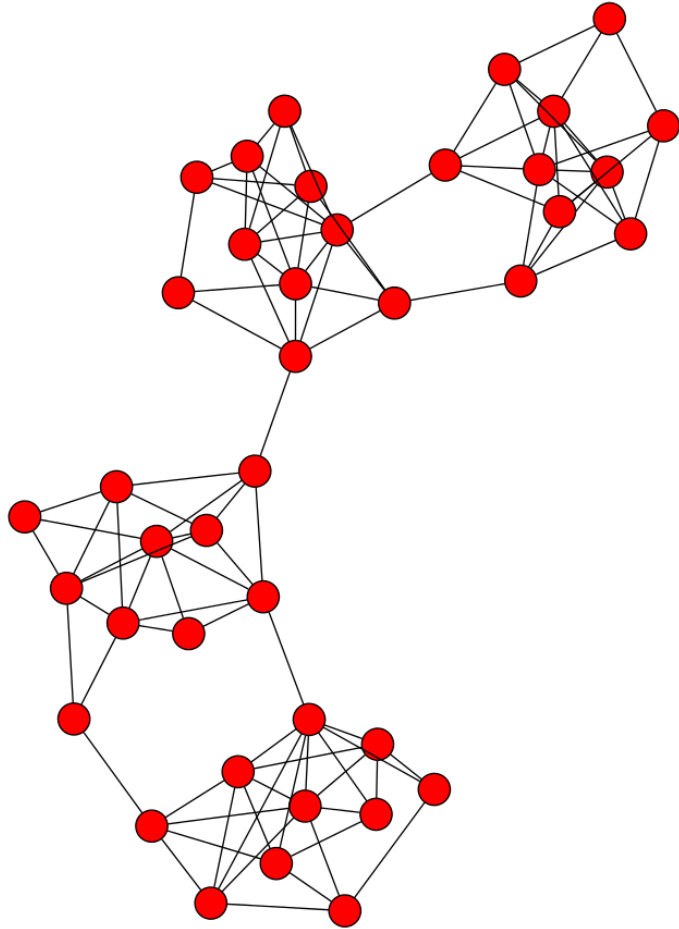
- **3) Grouping:**
 - Sort components of reduced 1-dimensional vector
 - Identify clusters by splitting the sorted vector in two
- **How to choose a splitting point?**
 - Naïve approaches:
 - Split at **0** or median value
 - More expensive approaches:
 - Attempt to minimize normalized cut in 1-dimension (sweep over ordering of nodes induced by the eigenvector)



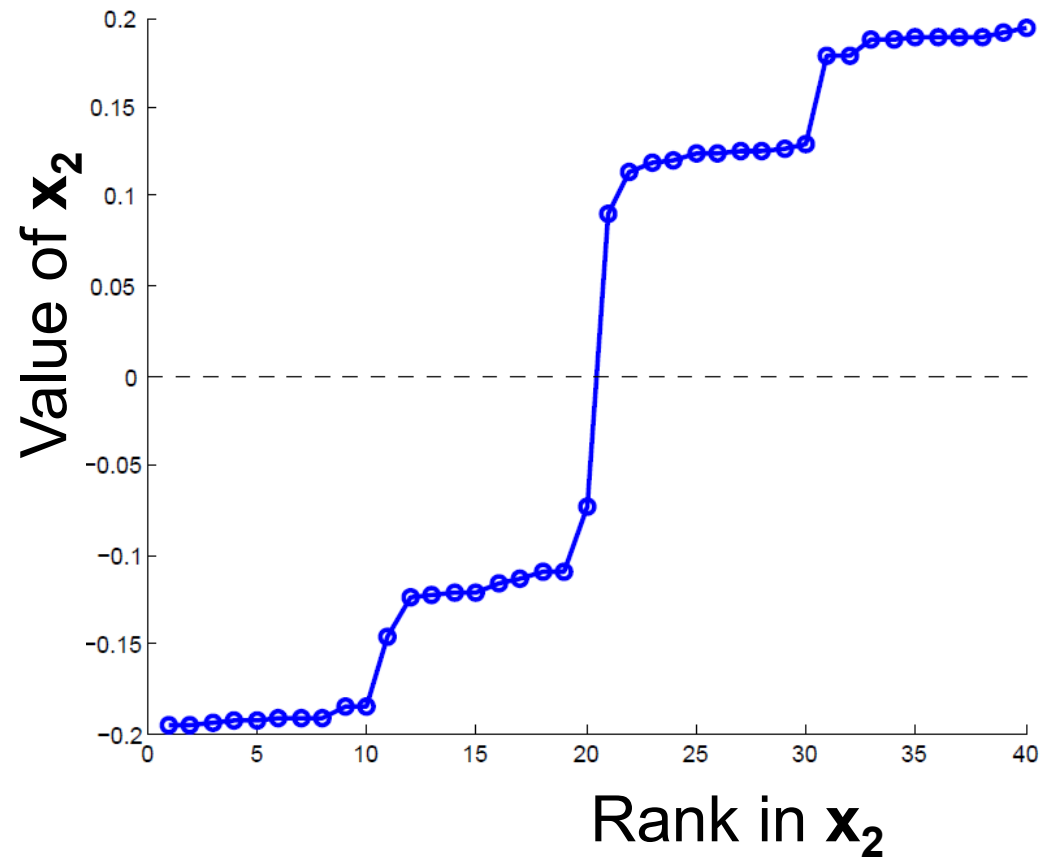
Example: Spectral Partitioning



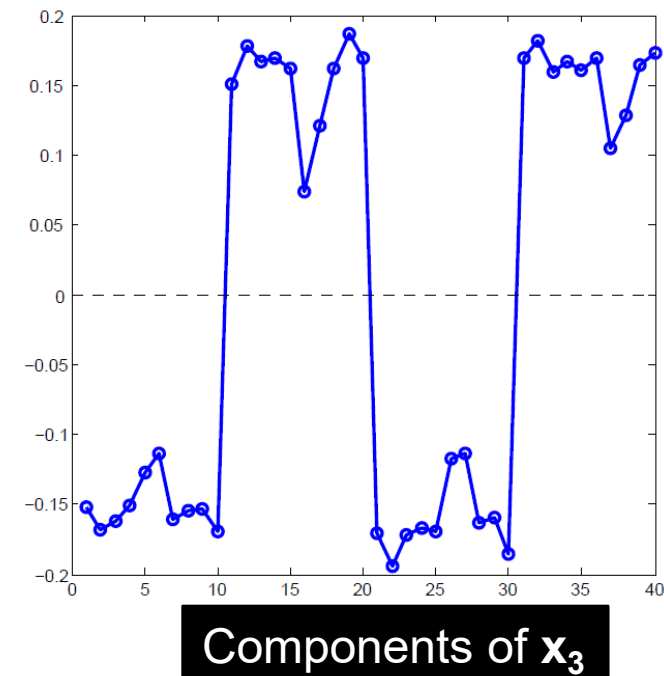
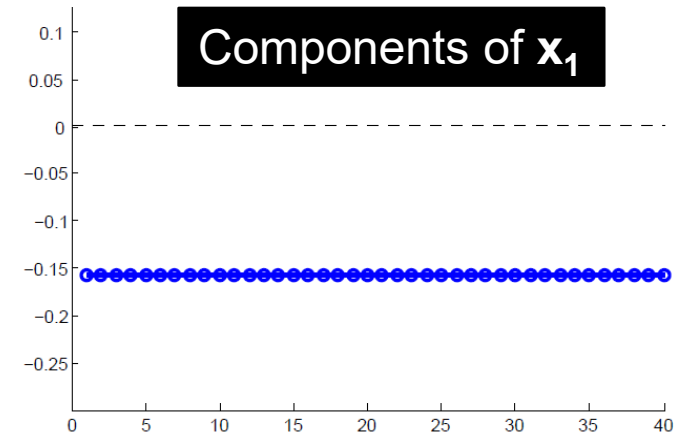
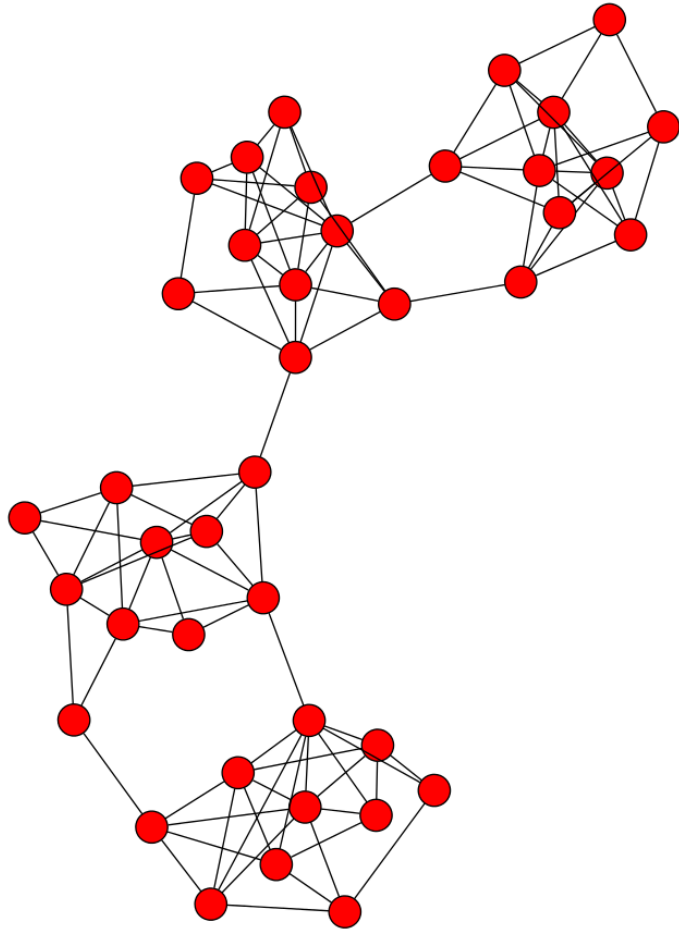
Example: Spectral Partitioning



Components of $\mathbf{x}_2$



Example: Spectral partitioning



k-Way Spectral Clustering

- How do we partition a graph into k clusters?
- Two basic approaches:
 - **Recursive bi-partitioning** [Hagen et al., '92]
 - Recursively apply bi-partitioning algorithm in a hierarchical divisive manner
 - Disadvantages: Inefficient, unstable
 - **Cluster multiple eigenvectors** [Shi-Malik, '00]
 - Build a reduced space from multiple eigenvectors
 - Commonly used in recent papers
 - A preferable approach...

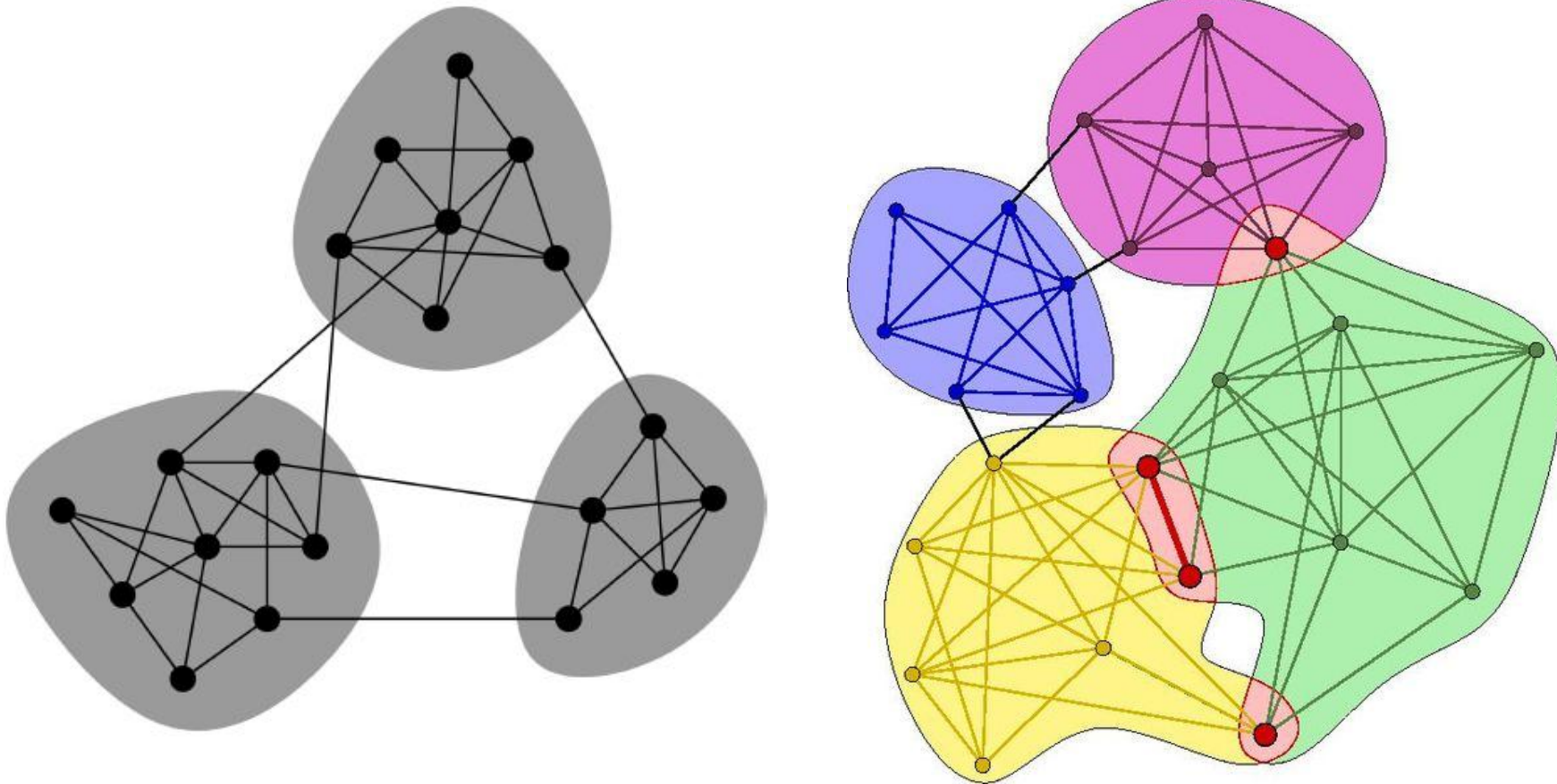
Why use multiple eigenvectors?

- **Well-separated space**
 - Transforms data to a new “embedded space”, consisting of k orthogonal basis vectors
- Multiple eigenvectors prevent instability due to information loss
- **Emphasizes cohesive clusters**
 - Associations between similar points are amplified, associations between dissimilar points are attenuated
 - The data begins to “approximate a clustering”

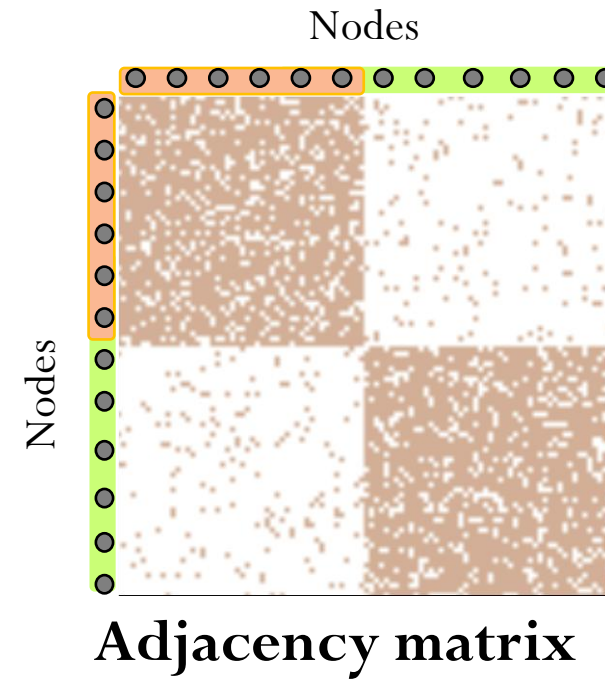
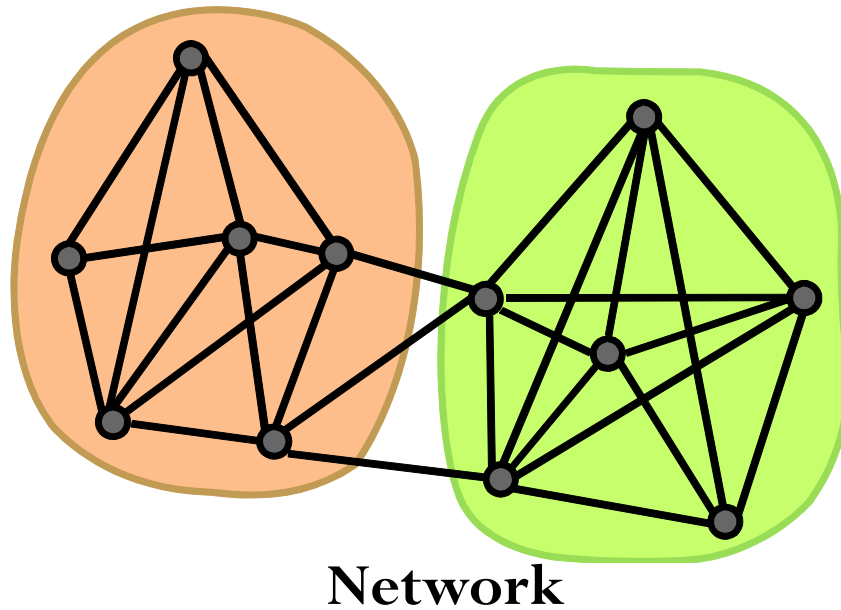
Overlapping communities

Overlapping Communities

- **Non-overlapping vs. overlapping communities**

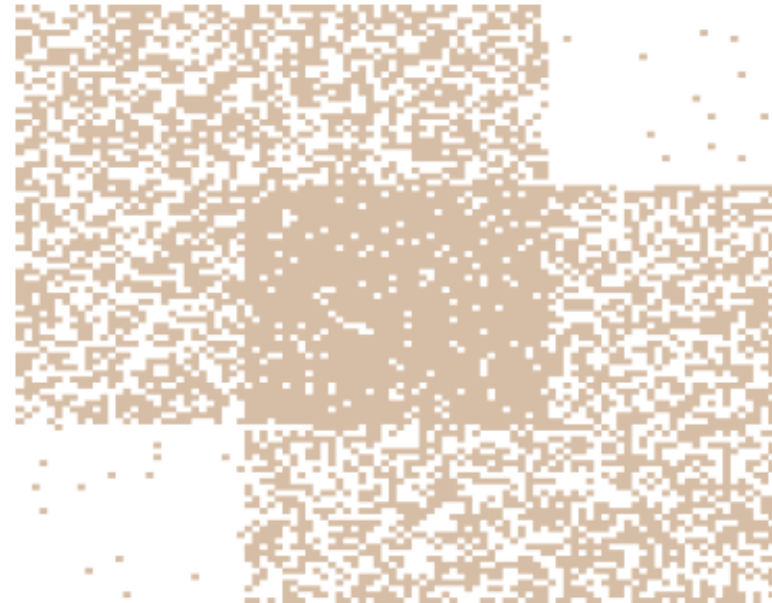
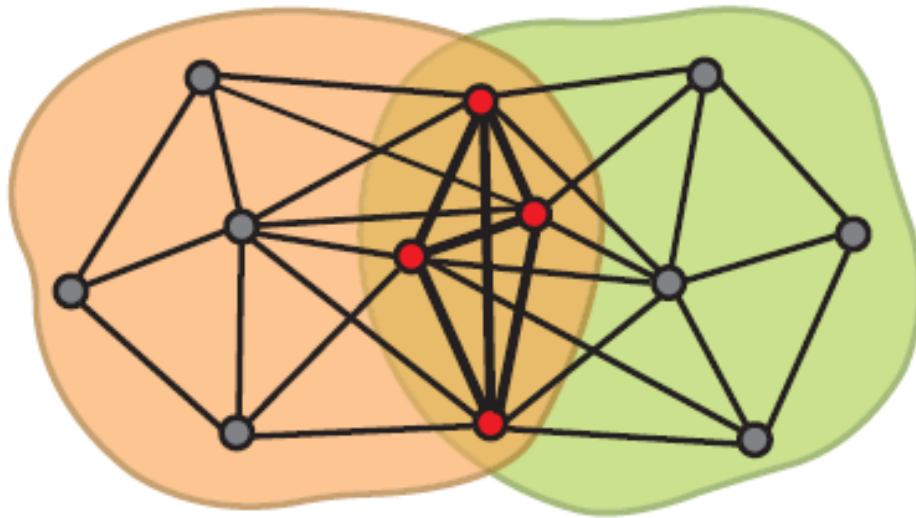


Non-overlapping Communities



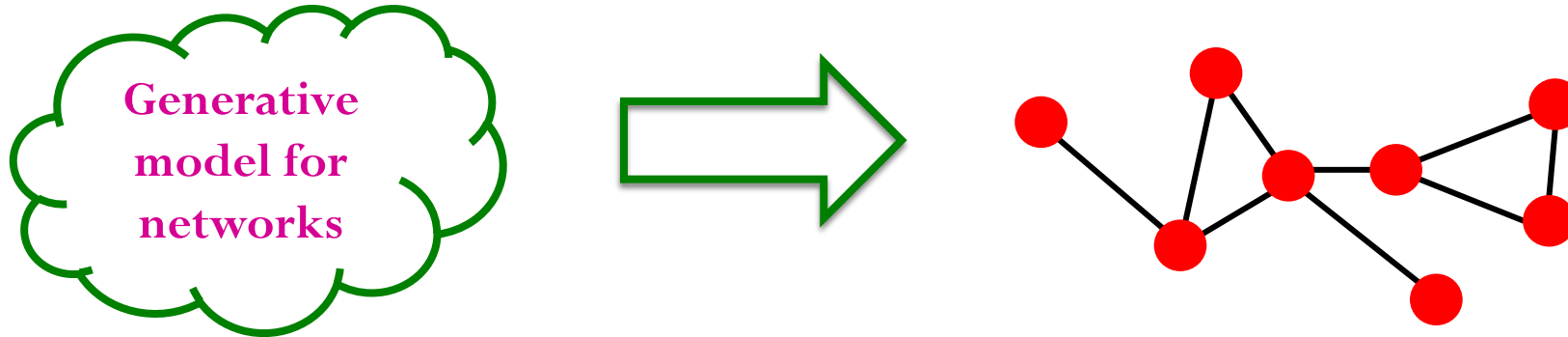
Communities as Tiles!

- **Overlapping communities:**
Edge density in the overlaps is higher!

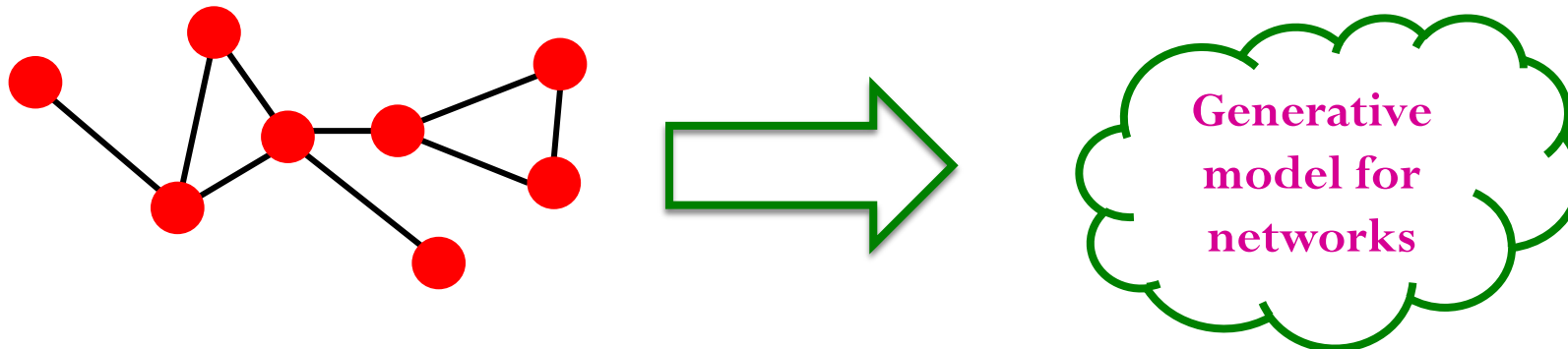


Plan of attack

- 1) Given a model, we generate the network:

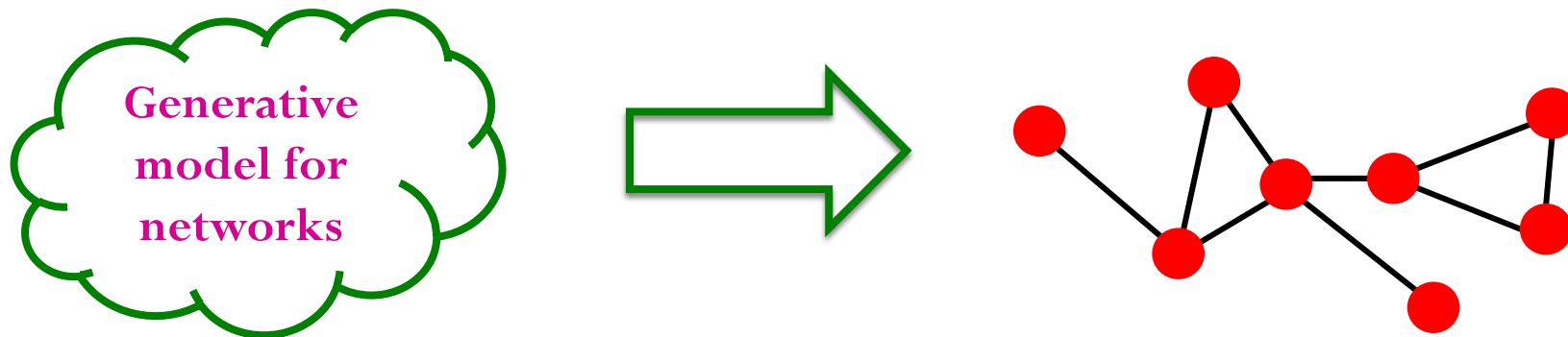


- 2) Given a network, find the “best” model



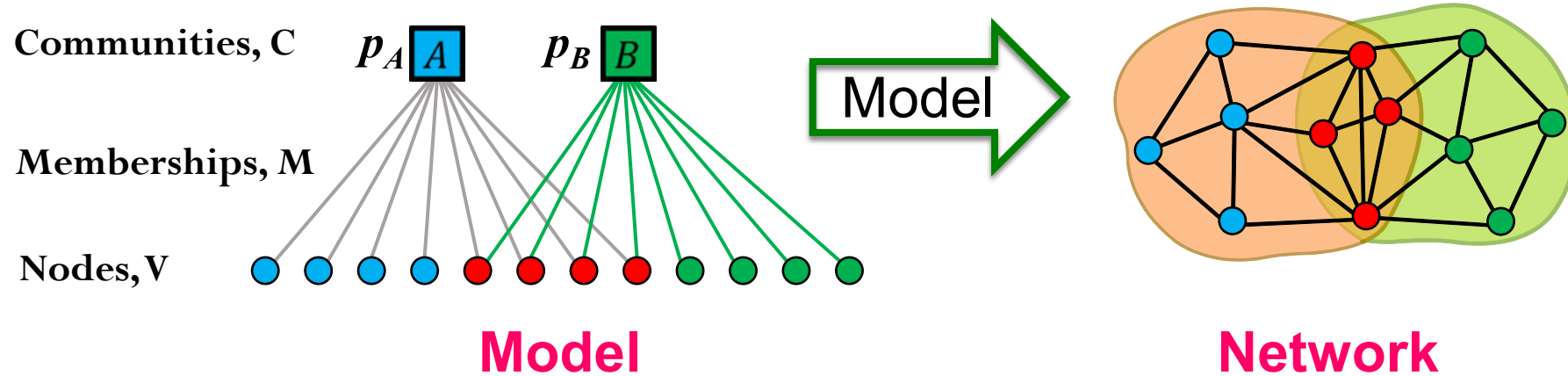
Model of networks

- **Goal:** Define a model that can generate networks
 - The model will have a set of “parameters” that we will later want to estimate (and detect communities)



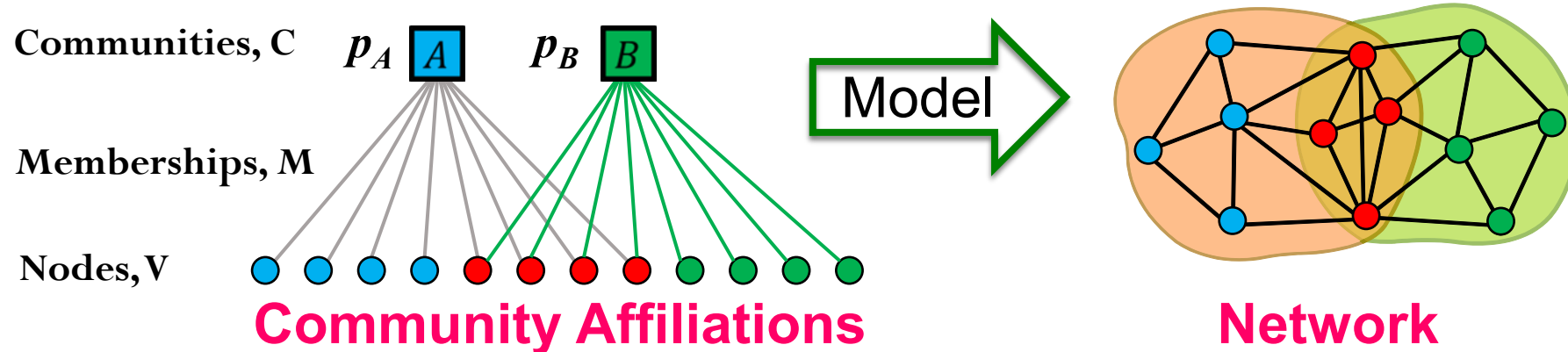
- **Q:** Given a set of nodes, how do communities “generate” edges of the network?

Community-Affiliation Graph



- **Generative model $B(V, C, M, \{p_c\})$ for graphs:**
 - Nodes V , Communities C , Memberships M
 - Each community c has a single probability p_c
 - Later we fit the model to networks to detect communities

AGM: Generative Process



- **AGM generates the links:**
 - For each pair of nodes in community **A**, we connect them with prob. p_A
 - The overall edge probability is:

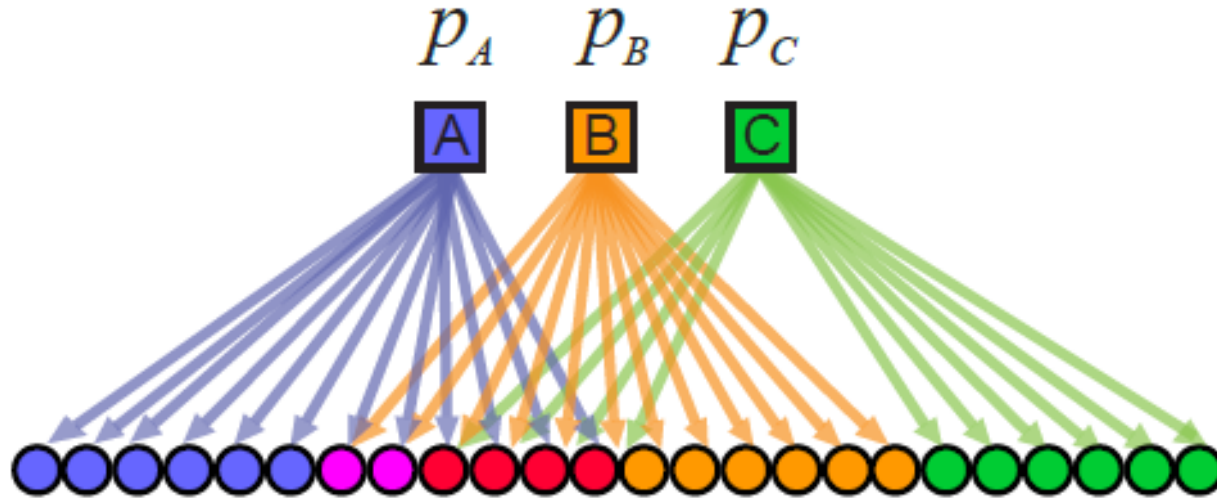
$$P(u, v) = 1 - \prod_{c \in M_u \cap M_v} (1 - p_c)$$

If u, v share no communities: $P(u, v) = \varepsilon$

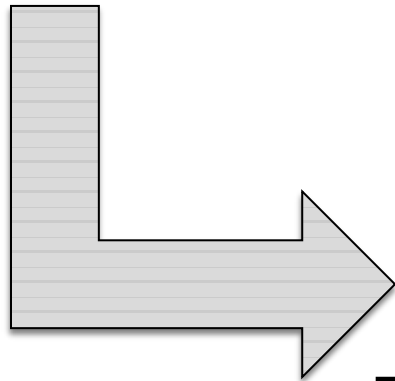
M_u ... set of communities
node u belongs to

Think of this as an “OR” function: If at least 1 community says “YES” we create an edge

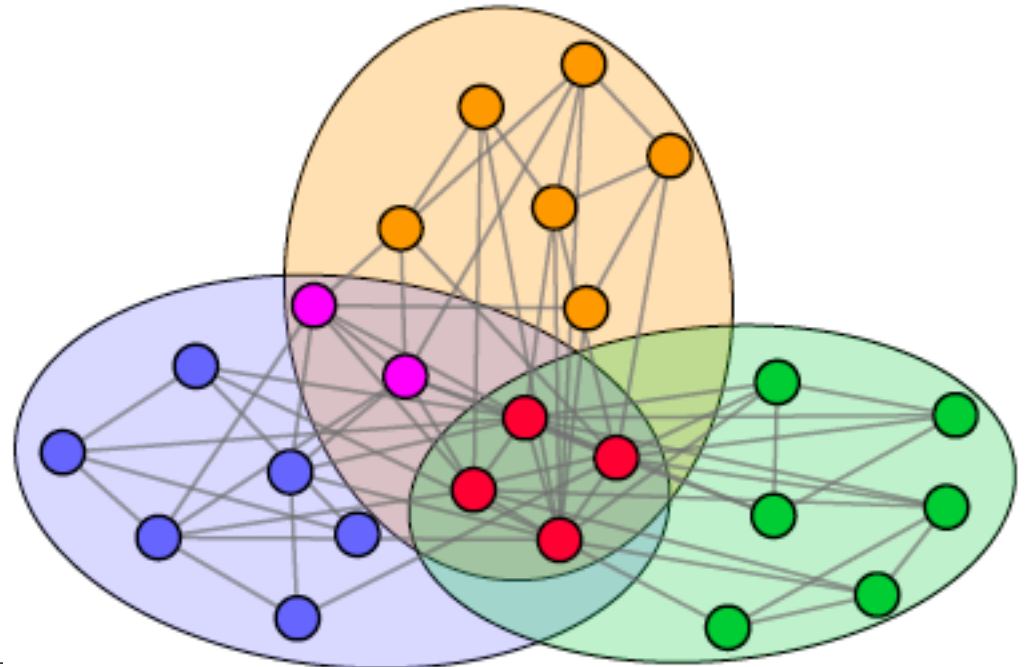
Recap: AGM networks



Model



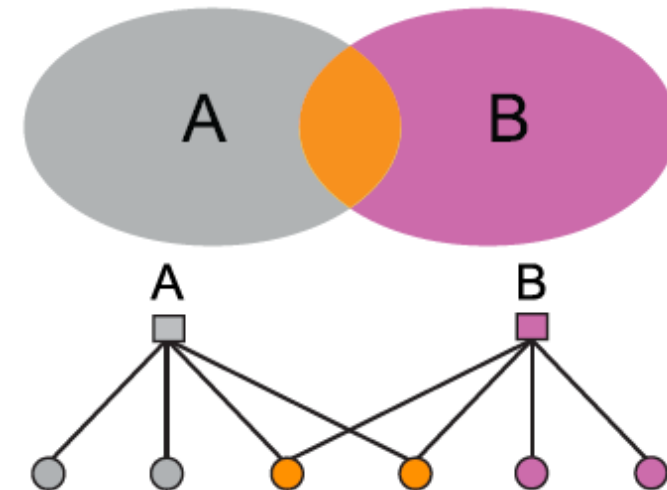
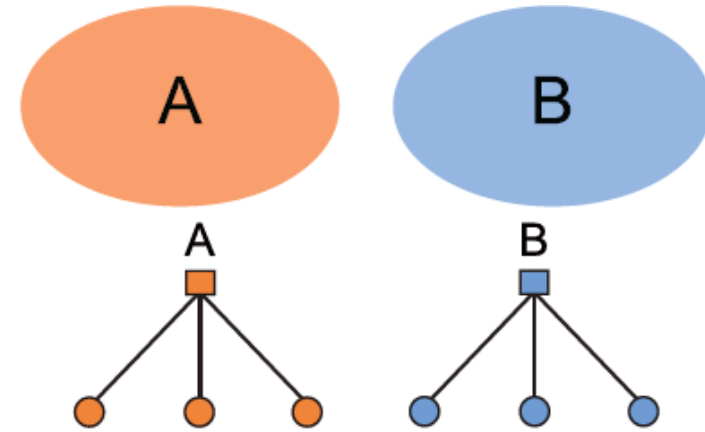
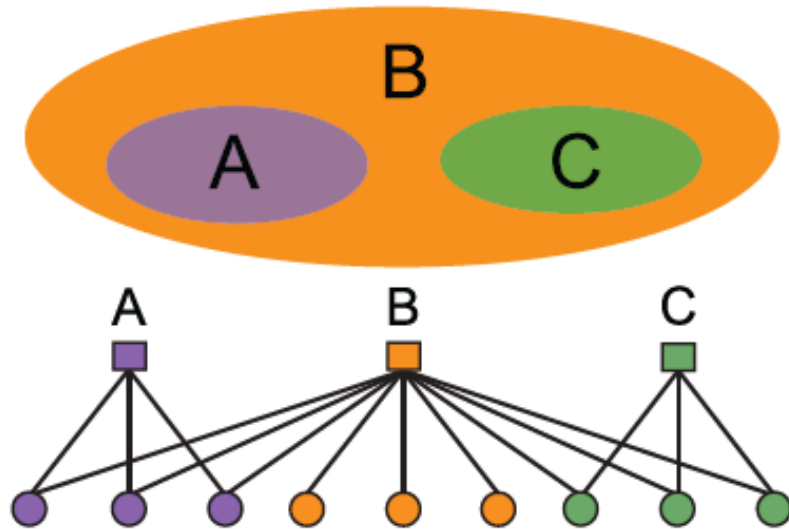
Network



AGM: Flexibility

- **AGM can express a variety of community structures:**

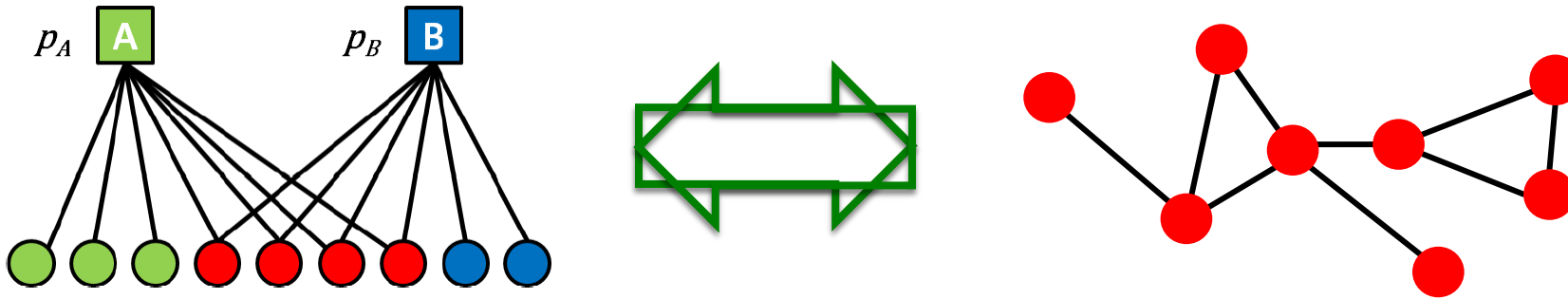
Non-overlapping, Overlapping,
Nested



How do we detect communities with AGM?

Detecting Communities

- Detecting communities with AGM:



Given a Graph $G(V, E)$, find the Model

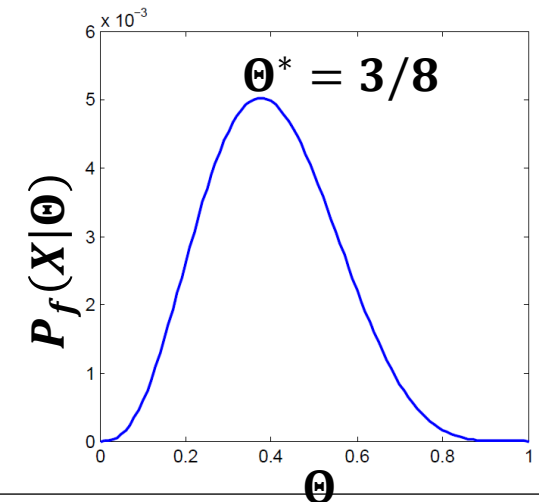
- 1) Affiliation graph M
- 2) Number of communities C
- 3) Parameters p_c

Maximum Likelihood Estimation

- **Maximum Likelihood Principle (MLE):**
 - **Given:** Data X
 - **Assumption:** Data is generated by some model $f(\Theta)$
 - f ... model
 - Θ ... model parameters
 - **Want to estimate $P_f(X|\Theta)$:**
 - The probability that our model f (with parameters Θ) generated the data
 - **Now let's find the most likely model that could have generated the data:** $\arg \max_{\Theta} P_f(X|\Theta)$

Example: MLE

- Imagine we are given a set of coin flips
- **Task:** Figure out the bias of a coin!
 - **Data:** Sequence of coin flips: $X = [1, 0, 0, 0, 1, 0, 0, 1]$
 - **Model:** $f(\Theta)$ = return 1 with prob. Θ , else return 0
 - **What is $P_f(X|\Theta)$?** Assuming coin flips are independent
 - So, $P_f(X|\Theta) = P_f(1|\Theta) * P_f(0|\Theta) * P_f(0|\Theta) \dots * P_f(1|\Theta)$
 - What is $P_f(1|\Theta)$? Simple, $P_f(1|\Theta) = \Theta$
 - Then, $P_f(X|\Theta) = \Theta^3(1 - \Theta)^5$
 - For example:
 - $P_f(X|\Theta = 0.5) = 0.003906$
 - $P_f(X|\Theta = \frac{3}{8}) = 0.005029$
 - **What did we learn?** Our data was most likely generated by coin with bias $\Theta = 3/8$



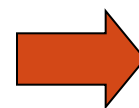
MLE for Graphs

- **How do we do MLE for graphs?**
 - Model generates a **probabilistic adjacency matrix**
 - We then flip all the entries of the probabilistic matrix to obtain the **binary adjacency matrix A**

For every pair
of nodes u, v
AGM gives the
prob. p_{uv} of
them being
linked

0	0.10	0.10	0.04
0.10	0	0.02	0.06
0.10	0.02	0	0.06
0.04	0.06	0.06	0

Flip
biased
coins



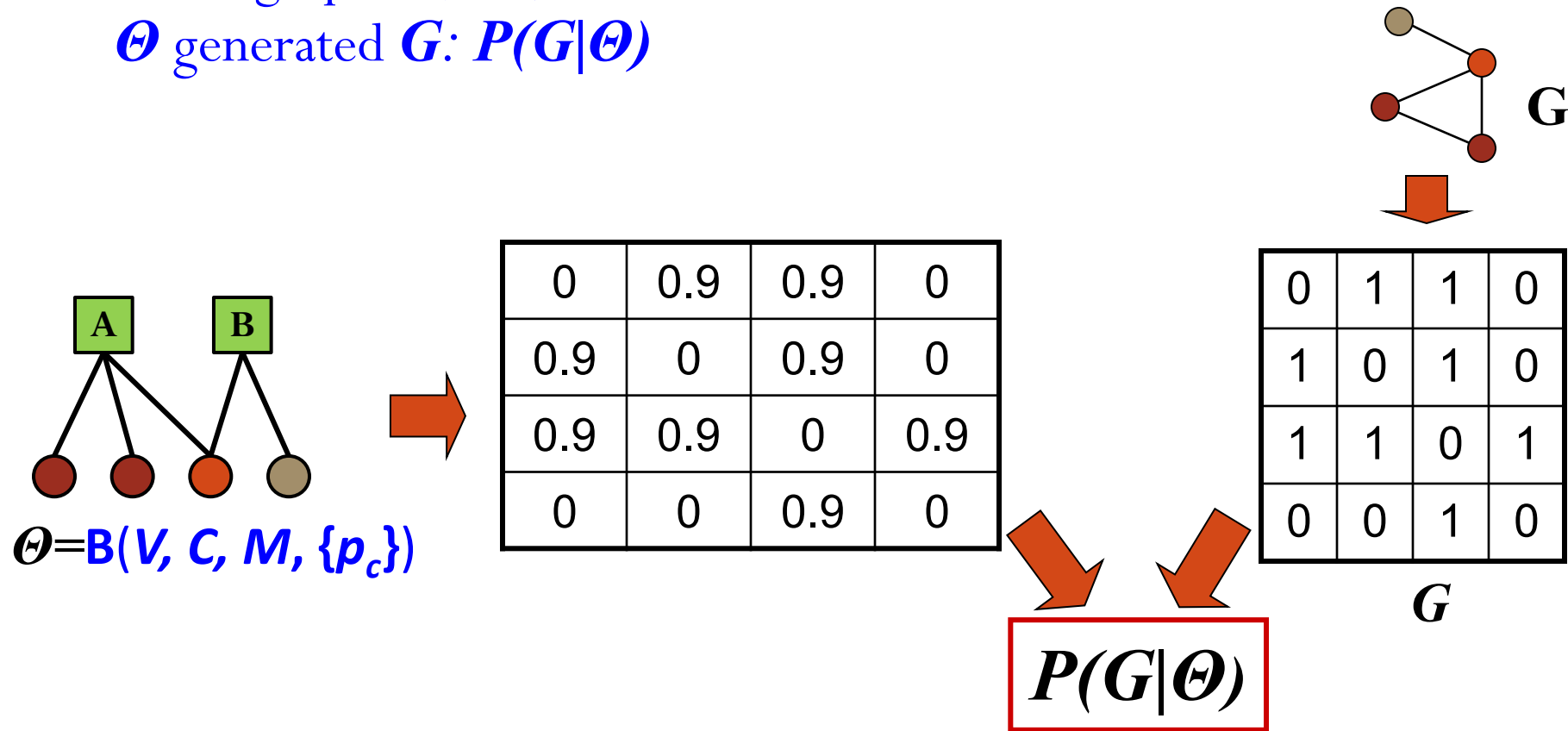
0	1	0	0
1	0	1	1
0	1	0	1
0	1	1	0

- **The likelihood of AGM generating graph G :**

$$P(G \mid \Theta) = \prod_{(u,v) \in E} P(u,v) \prod_{(u,v) \notin E} (1 - P(u,v))$$

Graphs: Likelihood $P(G|\Theta)$

- Given graph $G(V,E)$ and Θ , we calculate likelihood that Θ generated G : $P(G|\Theta)$



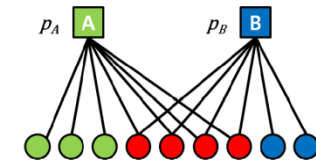
$$P(G | \Theta) = \prod_{(u,v) \in E} P(u,v) \prod_{(u,v) \notin E} (1 - P(u,v))$$

MLE for AGM

- Our goal is to find $B(V, \mathcal{C}, M, \{p_{\mathcal{C}}\})$ such that:

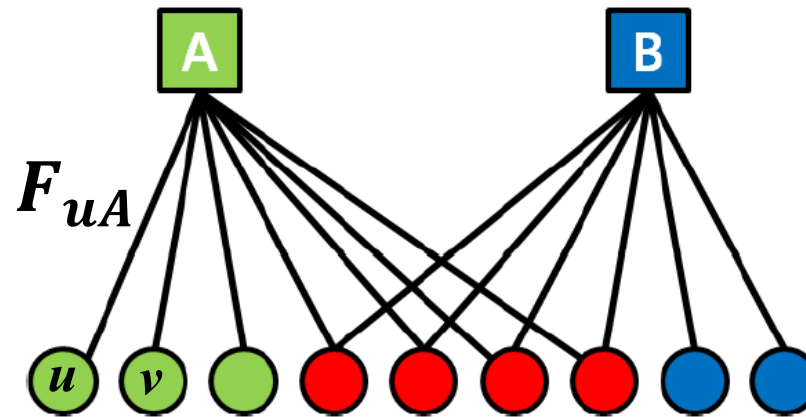
$$\arg \max_{B(V, \mathcal{C}, M, \{p_{\mathcal{C}}\})} \prod_{u, v \in E} P(u, v) \prod_{uv \notin E} (1 - P(u, v))$$

- **Problem:** Finding $\mathbf{B}$ means finding the bipartite affiliation network.
 - There is no nice way to do this.
 - Fitting $B(V, \mathcal{C}, M, \{p_{\mathcal{C}}\})$ is too hard, let's change the model (so it is easier to fit)!



From AGM to BigCLAM

- **Relaxation:** Memberships have strengths

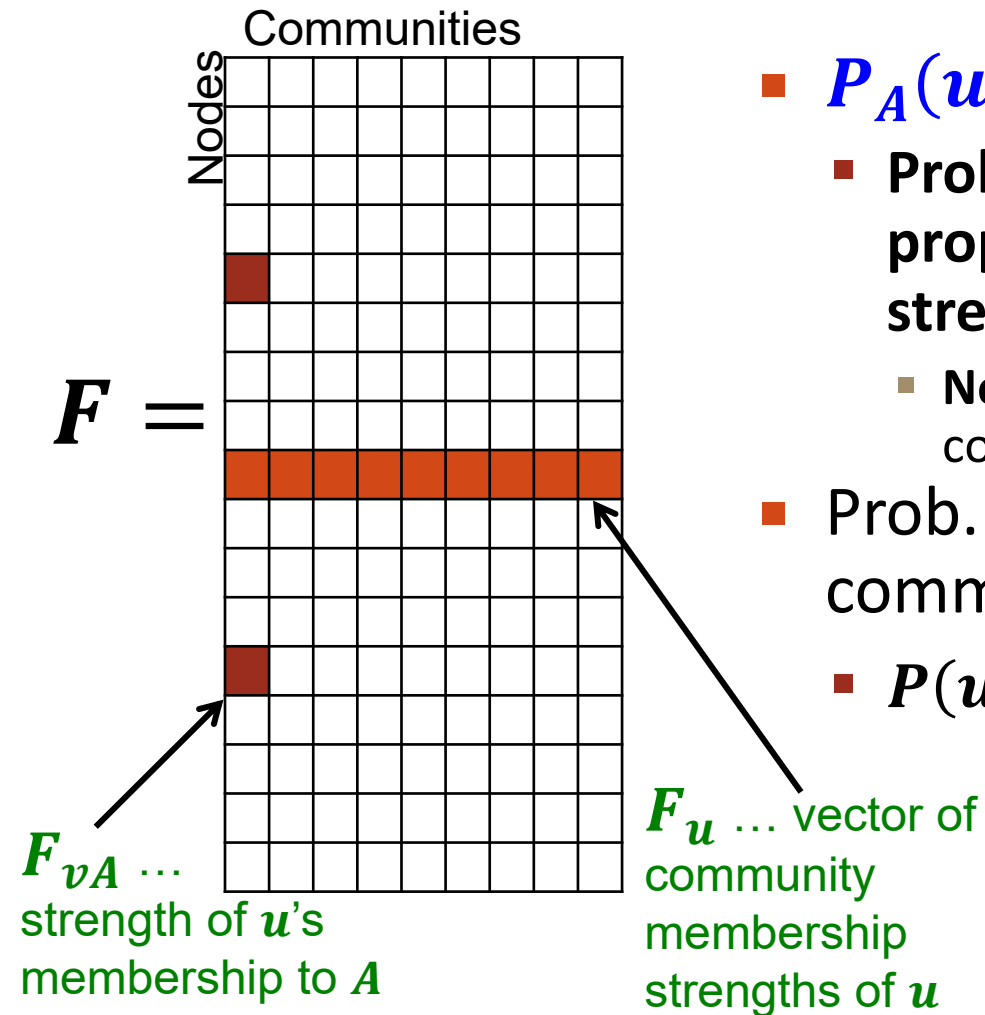


- F_{uA} : The membership strength of node u to community A ($F_{uA} = 0$: no membership)
- Each community A links nodes independently:

$$P_A(u, v) = 1 - \exp(-F_{uA} \cdot F_{vA})$$

Factor Matrix F

- Community membership strength matrix F



- $P_A(u, v) = 1 - \exp(-F_{uA} \cdot F_{vA})$
 - Probability of connection is proportional to the product of strengths
 - Notice: If one node doesn't belong to the community ($F_{uC} = 0$) then $P(u, v) = 0$
- Prob. that **at least one** common community C links the nodes:
 - $P(u, v) = 1 - \prod_C (1 - P_C(u, v))$

From AGM to BigCLAM

- Community **A** links nodes **u**, **v** independently:

$$P_A(u, v) = 1 - \exp(-F_{uA} \cdot F_{vA})$$

- Then prob. at least one common **C** links them:

$$\begin{aligned} P(u, v) &= 1 - \prod_C (1 - P_C(u, v)) \\ &= 1 - \exp(-\sum_C F_{uC} \cdot F_{vC}) \\ &= 1 - \exp(-F_u \cdot F_v^T) \end{aligned}$$

- Example **F** matrix:

$$F_u : \begin{bmatrix} 0 & 1.2 & 0 & 0.2 \end{bmatrix}$$

$$F_v : \begin{bmatrix} 0.5 & 0 & 0 & 0.8 \end{bmatrix}$$

$$F_w : \begin{bmatrix} 0 & 1.8 & 1 & 0 \end{bmatrix}$$

$$\text{Then: } F_u \cdot F_v^T = 0.16$$

$$\text{And: } P(u, v) = 1 - \exp(-0.16) = 0.14$$

$$\text{But: } P(u, w) = 0.88$$

$$P(v, w) = 0$$

Node community
membership strengths

BigCLAM: How to find F

- **Task:** Given a network $G(V, E)$, estimate F
- Find F that maximizes the likelihood:

$$\arg \max_F \prod_{(u,v) \in E} P(u, v) \prod_{(u,v) \notin E} (1 - P(u, v))$$

- where: $P(u, v) = 1 - \exp(-F_u \cdot F_v^T)$
- Many times we take the logarithm of the likelihood, and call it log-likelihood: $l(F) = \log P(G|F)$
- **Goal:** Find F that maximizes $l(F)$:

$$l(F) = \sum_{(u,v) \in E} \log(1 - \exp(-F_u F_v^T)) - \sum_{(u,v) \notin E} F_u F_v^T$$

BigCLAM: V1.0

$$l(F_u) = \sum_{v \in \mathcal{N}(u)} \log(1 - \exp(-F_u F_v^T)) - \sum_{v \notin \mathcal{N}(u)} F_u F_v^T$$

- **Compute gradient of a single row F_u of F :**

$$\nabla l(F_u) = \sum_{v \in \mathcal{N}(u)} F_v \frac{\exp(-F_u F_v^T)}{1 - \exp(-F_u F_v^T)} - \sum_{v \notin \mathcal{N}(u)} F_v$$

- **Coordinate gradient ascent:**
 - **Iterate over the rows of F :**
 - Compute gradient $\nabla l(F_u)$ of row u (while keeping others fixed)
 - Update the row F_u : $F_u \leftarrow F_u + \eta \nabla l(F_u)$
 - Project F_u back to a non-negative vector: If $F_{u\mathcal{C}} < \mathbf{0}$: $F_{u\mathcal{C}} = \mathbf{0}$
- **This is slow!** Computing $\nabla l(F_u)$ takes linear time in $|V|$!

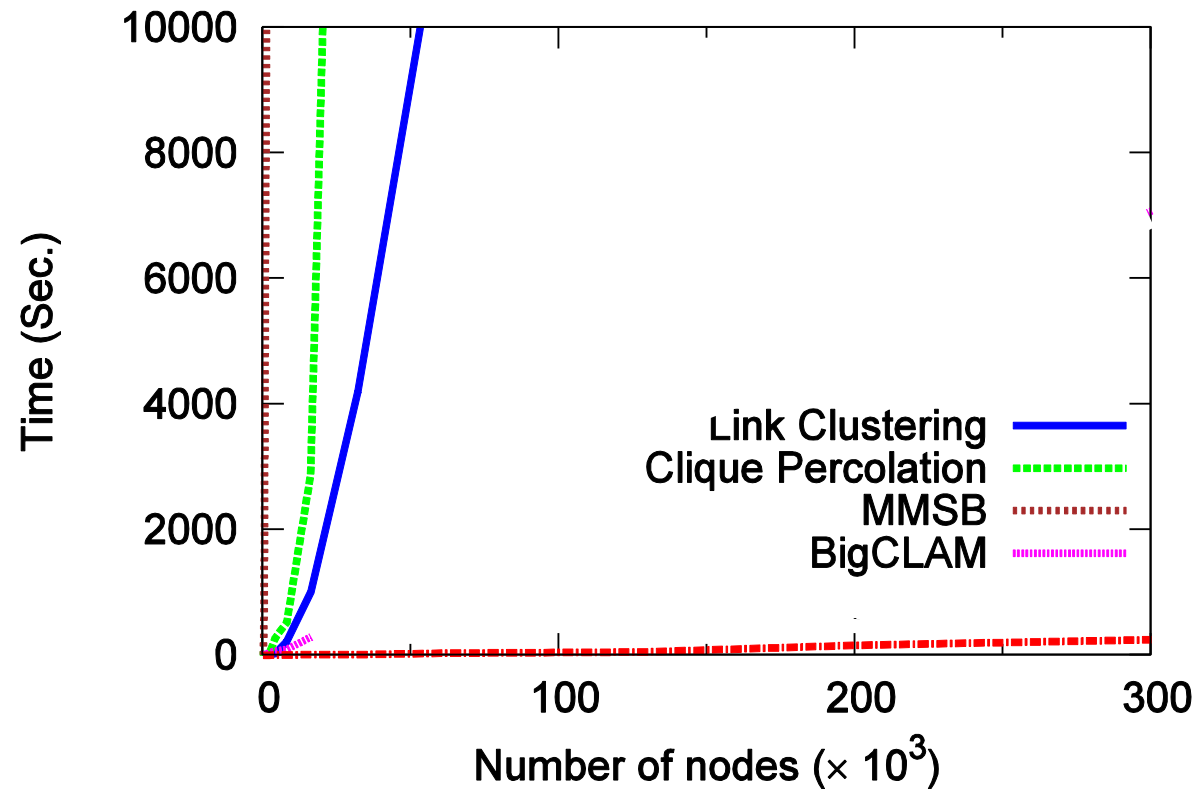
BigCLAM: V2.0

- However, we notice:

$$\sum_{v \notin \mathcal{N}(u)} F_v = \left(\sum_v F_v - F_u - \sum_{v \in \mathcal{N}(u)} F_v \right)$$

- We cache $\sum_v F_v$
- So, computing $\sum_{v \notin \mathcal{N}(u)} F_v$ now takes **linear time** in the degree $|\mathcal{N}(u)|$ of u
 - In networks degree of a node is much smaller than the total number of nodes in the network, so this is a significant speedup!

BigClam: Scalability



- **BigCLAM takes 5 minutes for 300k node nets**
 - Other methods take 10 days
- **Can process networks with 100M edges!**

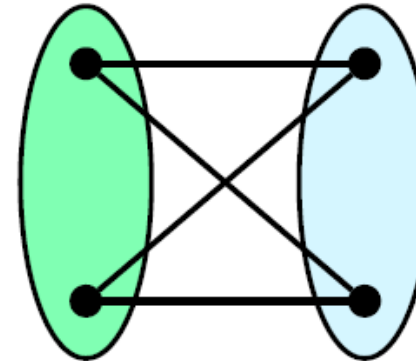
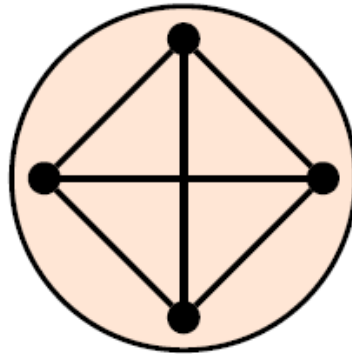
Extension: Directed memberships

Extension: Beyond Clusters

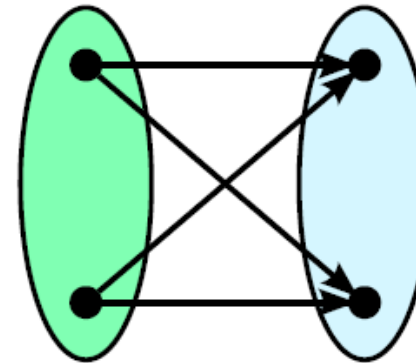
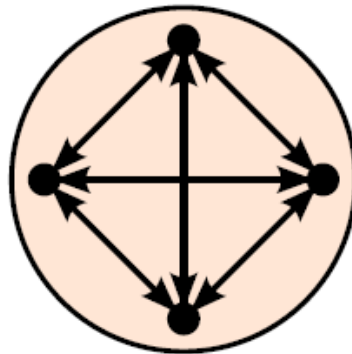
Cohesive

2-mode

Undirected



Directed

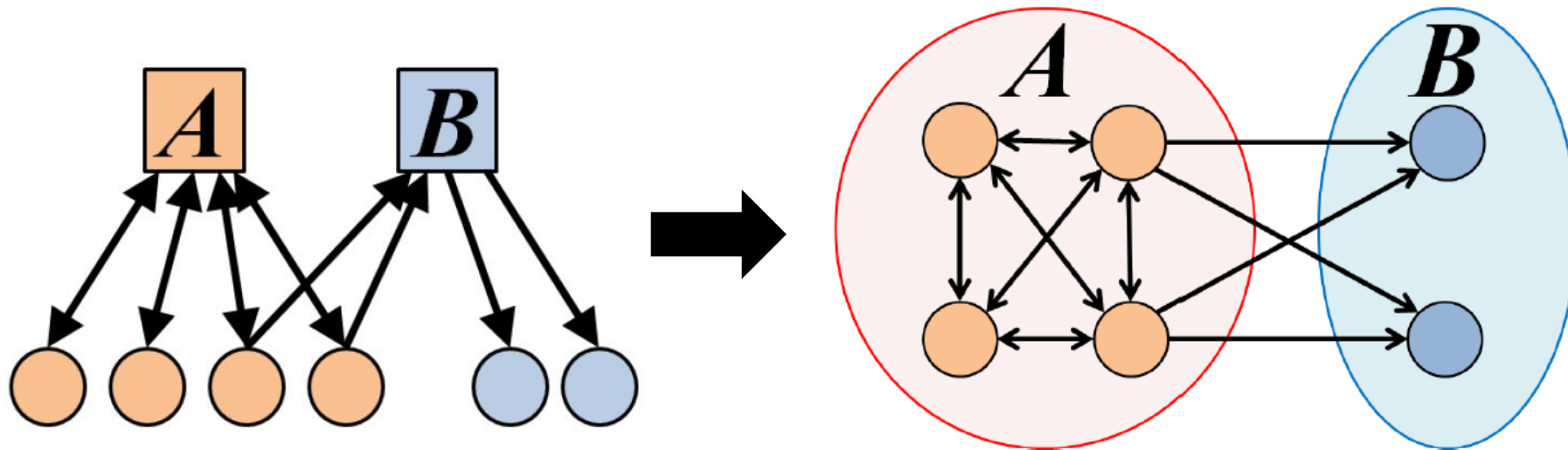


Extension: Directed AGM

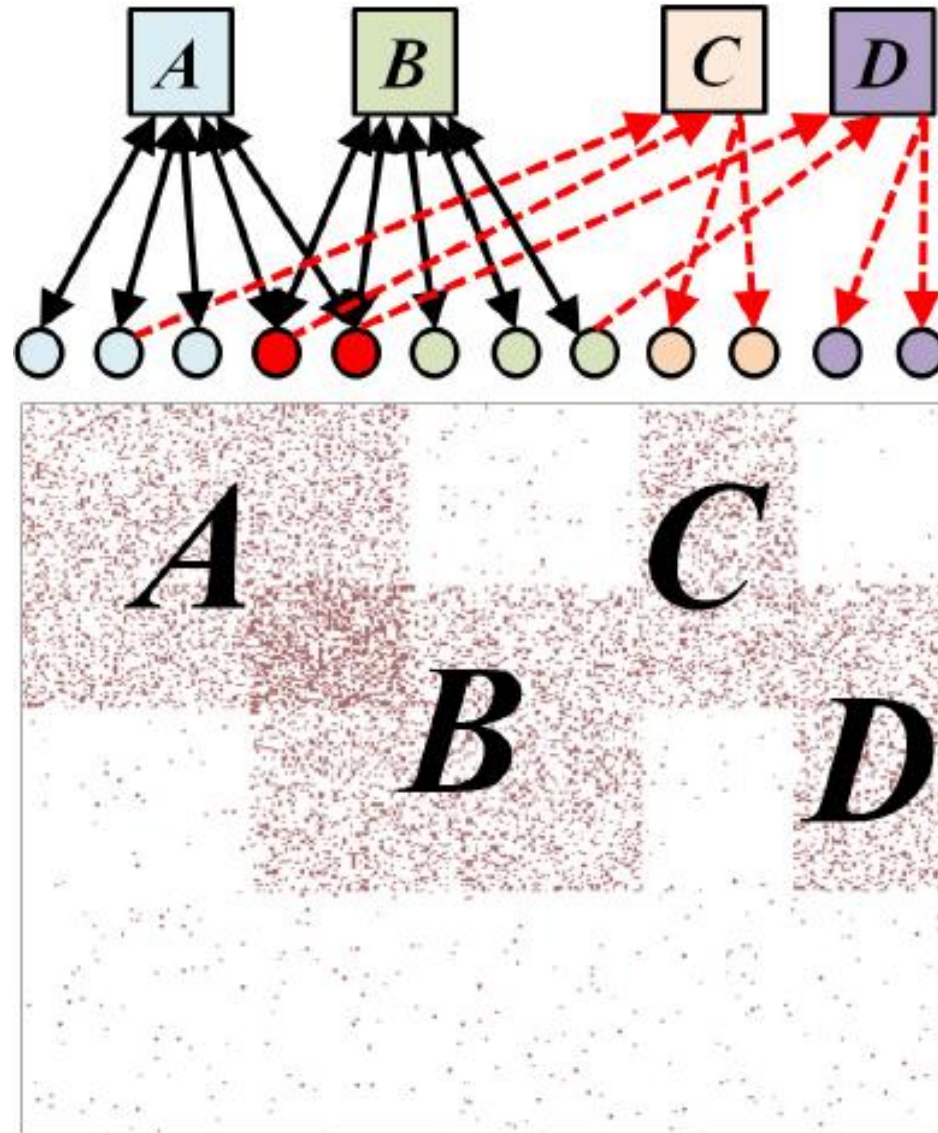
- **Extension:**

Make community membership edges directed!

- **Outgoing membership:** Nodes “**sends**” edges
- **Incoming membership:** Node “**receives**” edges

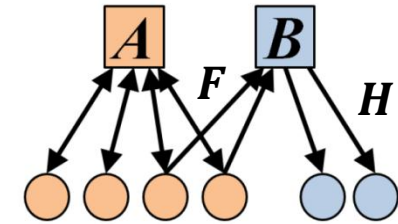


Example: Model and Network



Directed AGM

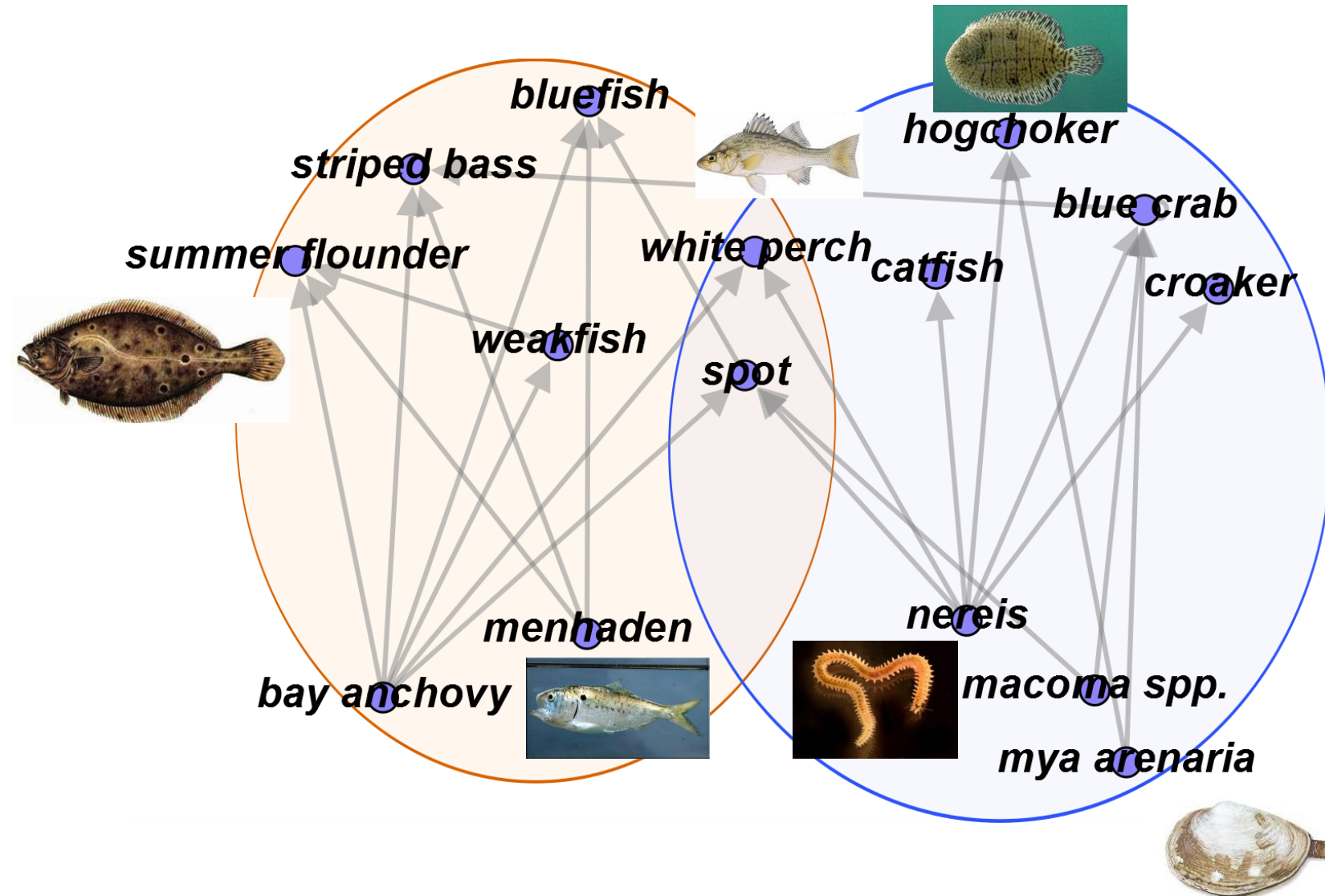
- **Everything is almost the same except now we have 2 matrices: F and H**
 - F ... out-going community memberships
 - H ... in-coming community memberships
- **Edge prob.: $P(u, v) = 1 - \exp(-F_u H_v^T)$**
- **Network log-likelihood:**



$$l(F, H) = \sum_{(u,v) \in E} \log(1 - \exp(-F_u H_v^T)) - \sum_{(u,v) \notin E} F_u H_v^T$$

which we optimize the same way as before

Predator-prey Communities



More details at...

- [Overlapping Community Detection at Scale: A Nonnegative Matrix Factorization Approach](#) by J. Yang, J. Leskovec. *ACM International Conference on Web Search and Data Mining (WSDM)*, 2013.
- [Detecting Cohesive and 2-mode Communities in Directed and Undirected Networks](#) by J. Yang, J. McAuley, J. Leskovec. *ACM International Conference on Web Search and Data Mining (WSDM)*, 2014.
- [Community Detection in Networks with Node Attributes](#) by J. Yang, J. McAuley, J. Leskovec. *IEEE International Conference On Data Mining (ICDM)*, 2013.